

Composing EVM Smart Contracts: A Seam Calculus

Standards as interfaces plus partial behavioural specifications,
and the assumption ledger of a composed system as a mathematical object

Zakaryae Boudi

Intelligence Economy Institute
research@tokenfrastructure.com

June 2026

Working paper / preprint — circulated for technical and academic feedback.

Abstract

Many high-impact failures in composed Ethereum Virtual Machine (EVM) applications arise not inside a single contract but at the *boundary* where one component silently assumes behaviour that a neighbour does not guarantee. We call such a boundary a *seam* and give it a precise calculus. We model a standard as a syntactic interface together with the behaviours the standard explicitly *requires*, and we model a component’s reliance at a seam as a separate set of *needs* that may be stronger than any interface or standard guarantees. We separate two phenomena the informal literature conflates: *specification conflict*, where two requirements are jointly unsatisfiable (rare, and decidable only within an explicitly chosen decidable logical fragment), and *seam non-conformance*, where a chosen implementation fails a neighbour’s need (common, and in general undecidable, which is why it is the proper target of heavyweight verification). For the safety fragment we instantiate a one-directional *soundness* theorem in the rely–guarantee tradition: if the assume–guarantee ledger is acyclic and every relied-upon need is discharged by a concrete guarantee or external attestation, then the composed system satisfies the conjunction of the *catalogued* seam guarantees. We are explicit about what this does *not* establish: it does not make smart-contract correctness decidable, and ledger well-formedness is necessary for a complete assurance argument but not sufficient for system safety, since internal bugs, mis-specified or missing properties, an incomplete threat model, and false attestations all lie outside the ledger. We then turn the calculus into practice along three axes. First, a dissection of the load-bearing EVM standards (tokens, approvals, vaults, signatures, accounts, flash loans, oracles, proxies) and the six structuring DeFi platforms (Uniswap v4, Aave V3, Compound III, Morpho Blue, Balancer V3, Curve NG), each read as a guarantee/need pair with its principal seam and a stated adversary. Second, a set of guarded design rules for constructing the discharge ledger. Third, a fully worked case study — a yield router stacking an ERC-4626 vault on Aave V3 behind a UUPS proxy with a Chainlink oracle, with a concrete normalising adapter — carried end to end: properties enumerated, seams identified, the ledger built, and two distinct faults (a token/vault non-conformance and a solvency/liveness cycle) mechanically surfaced. We connect the engineering layer to a companion formal substrate: a property catalogue with a five-axis classification and an assume–guarantee ledger. Our contribution is to make the *assumption ledger* — the audit artifact, distinct from the audit itself — a mathematical object: a typed, finite graph that separates decidable bookkeeping from specification entailment and implementation verification, makes hidden integration assumptions explicit, and localises the residual verification and trust burden rather than hiding it. We claim this for the ledger, not for the audit, which remains socio-technical.

Contents

1	Introduction	3
1.1	The problem	3
1.2	Why a calculus, and what we add over an informal survey	4
1.3	Contributions and structure	5
2	Primer: traces, properties, and assume–guarantee reasoning	6
3	The seam model	7
3.1	Standards as interface plus assumption set	7
3.2	Seams and the two faults of composition	8
3.3	Threat model	9
3.4	Compositional soundness (safety fragment)	10
3.5	Bounded liveness and economic needs	12
4	The structuring standards, dissected as guarantee/need pairs	13
4.1	Asset interfaces: what flows across a seam	13
4.2	Signatures and accounts: who may authorise across a seam	14
4.3	Cross-cutting primitives and architectural EIPs	14
4.4	Libraries as risk postures	15
5	The DeFi platforms, dissected by extension point	15
6	Guarded design rules for constructing the discharge ledger	16
6.1	Why upgradeability is decided first	16
6.2	The rules	17
6.3	Upgradeability: where the logic of change lives	17
6.4	Migration paths	18
7	Worked case study: a yield router, end to end	21
7.1	Step 1 — enumerate components and standards	21
7.2	Step 2 — enumerate properties and seams	21
7.3	Step 3 — build the discharge graph and find the fault	21
7.4	Attack-pattern reconstruction: the ERC-4626 inflation attack as a seam fault . . .	23
8	Formalisation: the assumption ledger as a mathematical object	24
8.1	The property of a standard	25
8.2	The three structural theorems, stated precisely	25
8.3	Soundness of the ledger argument, relative to a catalogue	25
8.4	The bridge to the verifiable-agent framework	26
8.5	The composition stack as system dependency graph	27

9	Making the ledger operational	28
9.1	The two faults, as a spine	28
9.2	A taxonomy of seam types	29
9.3	A canonical property, concretely	29
9.4	Evidence as a tuple, and why the class order is only a partial order	30
9.5	Economic needs, with a worked template	30
9.6	A minimal internal catalogue	31
9.7	An algebra of ledger operations	31
9.8	Reading the graph: proof-flow, not call-flow	32
9.9	An operational-leaf template	32
9.10	What the calculus cannot detect	33
9.11	Relation to existing audit methodologies	33
10	Related work	35
11	Limits and open questions	36

1 Introduction

1.1 The problem

A decentralised-finance (DeFi) application can range from a single self-contained contract to a deep assembly of interacting parts, and the interesting cases are the assemblies. Some protocols are deliberately minimal — a lone token, an automated-market-maker pair, a single-purpose vault — and may operate with few on-chain dependencies. But a large and economically dominant class of systems composes: a token contract, a vault that holds the token, a lending market the vault supplies into, an oracle that prices the collateral, a router that sequences the calls, a proxy that allows the code to be upgraded, and a governance contract that controls the proxy. These roles are not always all present, not always cleanly separable, and not always purely contractual: a system may collapse several roles into one contract, omit the proxy or the on-chain governance entirely, avoid routers or oracles, or — most importantly — depend on parties that no interface describes at all, such as third-party tokens, external oracle networks, shared liquidity pools, bridged assets, keeper bots, liquidators, front ends, and RPC providers. Many of the dependencies that matter are social, economic and infrastructural rather than written into an ABI [9]. *This paper concerns the composed case.* For a genuinely monolithic contract the calculus we develop degenerates to the classical single-contract verification problem, which is already well served; our contribution begins where contracts meet.

Some of these pieces are built to a published *standard* — an ERC (Ethereum Request for Comments) or an EIP (Ethereum Improvement Proposal) — but not all of them, and this unevenness is itself part of the problem. Fungible tokens (ERC-20) and tokenised vaults (ERC-4626) have genuine interface standards with broad adoption; routers, guards, adapters, hooks, governance modules, and oracle integrations are typically protocol-specific, or only loosely patterned after a standard, and carry no published interface contract at all. A component with no standard is not thereby exempt from the analysis; on the contrary, its behavioural expectations are entirely undocumented, which makes its boundaries *harder* to reason about, not easier. The promise of standards, where they exist, is composability: if every piece conforms to its interface, the pieces

should snap together.

They do snap together. The trouble is that an interface pins down the *shape* of an interaction more than its *behaviour*. A standard such as ERC-20 is not merely a list of names and types — it does say that **transfer** moves tokens and emits a **Transfer** event — but it leaves enough semantic room that conforming to the interface does not guarantee the properties downstream protocols routinely assume: that the recipient receives *exactly* the amount sent (violated by fee-on-transfer tokens), that balances are conserved between transfers (violated by rebasing tokens), that no extra code runs as a side effect (the ERC-777 lineage adds a **tokensReceived** hook by design, a reentrancy surface for any caller that did not expect it), and that there are no blacklists, pause switches, or unusual return-value conventions. Note the asymmetry of fault here: a fee-on-transfer or hook-bearing token is not “broken” — it may expose the ERC-20 ABI faithfully and behave exactly as its own design intends. The unsafety is created by an *integrator* that was written with narrower mental assumptions than the standard actually licenses. A vault written assuming “the amount I receive equals the amount sent” is correct in isolation and becomes unsafe the moment it is composed with a fee-on-transfer token — even though both may conform at the ABI level, while the integrator’s *unstated* assumption is what is violated.

We do not claim composition faults are *the* dominant failure mode — that would require a root-cause classification of incidents into seam versus non-seam that we have not performed, and indeed the largest single loss category in recent industry data is access control, which is frequently intra-contract [10]. The defensible claim is narrower: a large share of high-impact incidents involve cross-contract, dependency, oracle, callback, or upgrade interactions, and these are exactly the cases an interface-level analysis misses. Industry rankings list price-oracle manipulation, flash-loan-facilitated attacks, unchecked external calls, and proxy/upgradeability flaws among the top categories, and emphasise that exploits increasingly *chain* several of these within one atomic transaction [10, 8, 12]. Systematisations of DeFi independently identify permissionless composability as simultaneously the ecosystem’s principal source of innovation and of systemic fragility, with failures able to propagate across protocols faster than human intervention can respond [9]. For this class of failure the locus of risk sits not inside one contract but at the *boundaries between contracts* — the routers, hooks, adapters, guards, module registries, flash-loan callbacks, oracles, and proxies. We call such a boundary a *seam*, and the thesis of this paper is that for composed systems the seam is a productive unit of both design and audit. We present the seam calculus as a way to model this class, not as a claim that all smart-contract risk reduces to seams.

1.2 Why a calculus, and what we add over an informal survey

Calling boundaries “where the risk is” is a useful slogan but not yet a method. It also helps to be explicit about the four levels at which one might make a claim, because they have very different evidential burdens: (i) a *descriptive* claim that DeFi systems are composed; (ii) a *diagnostic* claim that many failures occur at seams; (iii) a *methodological* claim that a seam ledger helps audit those failures; and (iv) a *formal* claim relating ledger conditions to system properties. This paper advances all four but is careful that the fourth is a one-directional soundness statement over a chosen catalogue, not a decision procedure for security. To make the slogan a method we supply three things an informal survey does not.

1. **A precise model of what crosses a seam, and what it means for a seam to be discharged.** We model a standard as a pair $(\text{Iface}(X), \text{Spec}(X))$ — a syntactic interface and the behaviours the standard explicitly requires — and, separately, a component’s *need* $\text{Need}(C, \rho)$ at a seam, which is what *that integrator* relies on and may exceed any standard guarantee. A seam is *locally discharged* when an evidenced guarantee entails the relied-upon need under the stated trace layer and threat model; the supporting result, when

these local discharges compose, is a one-directional rely–guarantee soundness theorem (Theorem 3.1) [4, 6, 7]. Critically, we distinguish *specification conflict* from *seam non-conformance* (Section 3.2); the informal literature routinely conflates them, which mislocates the field’s flagship failure example.

2. **An explicit adversary.** Composition faults are exploited, not merely encountered. We state a threat model (Section 3.3) with concrete, composable capabilities — transaction ordering and reentrancy [8], flash liquidity, oracle manipulation, governance/upgrade capture, adversarial components, and a cross-domain/bridge capability — the ordering case being the maximal-extractable-value surface quantified by Daian et al. [11]; we tag each seam with the capability that makes its undischarged need dangerous.
3. **An honest account of decidability.** We separate the genuinely decidable part of the assurance argument (the ledger graph is acyclic and well-formed; specification entailment *within a chosen decidable fragment*) from the part that is undecidable in general (whether a deployed contract’s runs lie inside a property), and we show how the logical *shape* of each obligation dictates which verification tool can even attempt it (Theorem 8.2) [1, 3]. Claims of “mechanised audit” that elide this distinction overreach; we make the residue explicit and name it.

1.3 Contributions and structure

Section 2 is a self-contained primer on the formal vocabulary (traces, properties, safety/liveness, hyperproperties, assume–guarantee) for readers outside formal methods. Section 3 defines the seam model, the standard-versus-need terminology, and the two-fault distinction, states the threat model, and proves the one-directional compositional soundness theorem. Section 4 dissects the standards and Section 5 the DeFi platforms, each as a guarantee/need pair with its seam and adversary; the factual spine (versions, standardisation stages, the EIP-7702 status) is verified against primary sources. Section 6 gives the guarded design rules for constructing the ledger. Section 7 is the end-to-end worked case study with a concrete adapter. Section 8 connects everything to the companion property catalogue and states the catalogue-relative soundness result with an explicit “what this does not prove.” Section 9 makes the ledger operational — evidence semantics, seam taxonomy, a machine-readable property schema, a minimal internal catalogue, severity scoring, and explicit blind spots. Section 10 situates the work in the verification and composition literature. Section 11 states the limits, including the irreducible trust in external attestations.

What is new, and what is not. We are explicit that the soundness theorem (Theorem 3.1) is *not* a new result: it is an instantiation of classical assume–guarantee reasoning [4, 6, 7], and we use it only as a legitimising backbone. The contribution is the *instantiation layer*: (i) a typed seam model for EVM composition with the **lface/Spec/Need** separation; (ii) the specification-conflict versus seam-non-conformance distinction with its different decidability and remedies; (iii) a ledger structure connecting needs, guarantees, evidence classes, verification modalities, and adversary tags; (iv) a property-catalogue schema and a small algebra of ledger operations (Section 9.7); and (v) a workflow that makes audit assumptions explicit and assigns each to the right verification modality. The novelty is the disciplined representation, not a new theorem.

Status of this version. This is a working-paper version. It specifies the artifact structure — the property and ledger schemas (Section 9.3), the well-formedness checks (Section 9.7), and the adapter invariant (Section 7) — but does not yet include the executable checker or the Foundry invariant suite; building that artifact is the stated next step (Section 11).

2 Primer: traces, properties, and assume–guarantee reasoning

This section fixes vocabulary. A reader fluent in formal methods may skim to Section 3; a reader from the smart-contract world but not from verification should read it in full, because every later theorem is phrased in this language.

Executions as traces, and the layers of the alphabet. Fix a finite or countable *alphabet* A of observable atoms. An *execution* or *trace* is a finite or infinite sequence $\tau \in A^{\leq\omega}$: the history of everything observable as the system runs. We write $\Sigma = A^{\leq\omega}$ for all conceivable traces and $\text{Tr}(S) \subseteq \Sigma$ for those a system S can produce. A system *is* the set of its possible histories; this behavioural view lets us compare a contract against a requirement without inspecting its source.

The content of A is not innocent: many seam needs cannot even be *stated* without enough structure in the alphabet, and a property proved over too thin an alphabet is a proof about the wrong object. We therefore stratify A into layers and are explicit about which layer a need lives in (Table 1). The safety theorem of Section 3 is stated over whichever layers a given need references; crucially, the canonical EIP-7702 hazard — that `tx.origin == msg.sender` no longer implies “no programmable execution path” — is inexpressible in the EVM-local layer and requires the transaction-context layer, and the bridge/finality needs require the cross-domain layer.

Layer	Atoms it adds	Example need it can state
EVM-local	balances, storage slots, allowances, internal calls, reverts, logs, gas	balance conservation; no-overspend
Transaction-context	<code>msg.sender</code> , <code>tx.origin</code> , <code>delegatecall</code> context, EIP-7702 delegation/auth-list, transient storage	“caller has no programmable path”; reentrancy guards
Economic	oracle observations, prices, available liquidity, flash-loan availability	manipulation cost > extractable value; bounded slippage
Governance	proposals, timelock delays, role changes, upgrades	only timelock upgrades; bounded admin authority
Cross-domain	source message, bridge proof, finality depth, replay domain, destination execution	message authenticity; replay protection; bounded finality latency

Table 1. Trace layers. A need is only expressible if the alphabet carries the relevant atoms; a guarantee proved over a thinner layer than the need requires is a category error (Remark 8.2, gap on faithfulness). We tag each catalogued property with its layer.

We write $\Sigma = A^{\leq\omega}$ for the set of all conceivable traces over the chosen layers and $\text{Tr}(S) \subseteq \Sigma$ for those a system S can produce.

Properties as sets of traces. A *trace property* is simply a set $\Pi \subseteq \Sigma$ of “acceptable” histories. A system *satisfies* Π , written $S \models \Pi$, when every history it can produce is acceptable: $\text{Tr}(S) \subseteq \Pi$. “Balances are conserved across every transfer” names the set of all traces in which no transfer step changes the sum of balances; a rebasing token can produce a history outside that set, so it does not satisfy the property. The view is deliberately extensional: a property does not care *how* a system computes, only *which* histories result.

Safety and liveness. Two shapes of property recur. A *safety* property says “nothing bad ever happens”: it is violated, if at all, at a finite point (some prefix already witnesses the breach),

and once violated it cannot be repaired. “The vault is never insolvent” is safety: insolvency at any reachable state is a finite witness of violation. A *liveness* property says “something good eventually happens”: no finite prefix can refute it, because the good thing might still come later. “Every queued asynchronous withdrawal is eventually settled” is liveness. The distinction is not stylistic. Safety is checkable by watching for a bad prefix; liveness is not refutable by any finite observation, so testing and single-run fuzzing — which observe finite prefixes — can find safety violations but can never *establish* liveness. The Alpern–Schneider theorem (Theorem 8.1) [1, 2] makes this precise: topologically, safety properties are the closed sets and liveness properties the dense sets of a natural topology on Σ , and every property is the intersection of one of each.

Hyperproperties. Some requirements are not about a single history but about *relationships among histories*. “User A’s actions cannot influence what User B observes” (non-interference) compares pairs of traces; “the router behaves identically regardless of transaction order” compares a trace with its reorderings. Such a requirement is a *hyperproperty*: a set $\mathbb{H} \subseteq \mathcal{P}(\Sigma)$ of *sets* of traces [3]. Clarkson and Schneider’s classification places the practically important cases in the *k-safety* fragment: a violation is witnessed by a finite combination of k finite trace-prefixes. A *k-safety* property is checkable, but only by a method that can relate k runs at once — single-run tools are structurally blind to it. This is the formal reason that information-flow and order-independence obligations demand relational tooling, not fuzzing.

Assume–guarantee reasoning. When a system is built from interacting parts, one rarely proves the whole monolithically. Instead each part C is given a *contract*: “*assuming* the environment behaves like A_C , C *guarantees* G_C .” Composition is sound when the parts’ assumptions are met by their neighbours’ guarantees, with one subtlety that has tripped up the field since the 1990s: the reasoning must not be instantaneously circular. If C_1 assumes what C_2 guarantees and C_2 assumes what C_1 guarantees, naive conjunction is *unsound* — each could be vacuously “correct” only because it relies on the other, with neither ever doing its job. Abadi and Lamport’s *Composition Theorem* [6] resolves this by requiring that guarantees be *safety* properties discharged with a one-step delay (a component’s guarantee at time t may assume the environment only up to $t - 1$), or equivalently that the dependency be well-founded. We use exactly this discipline in Theorem 3.1; the well-foundedness appears as acyclicity of a discharge graph.

Remark 2.1 (Why the delay matters, concretely). A vault’s solvency guarantee may rely on the oracle’s price-validity assumption, and the oracle’s liveness may rely on the vault’s keeper calling it — a cycle. The Abadi–Lamport discipline forbids closing such a loop without a temporal stratification: someone must guarantee something *unconditionally first*. In our ledger this is the requirement that every dependency chain bottoms out at an *axiom* (a protocol-level fact) or an *external attestation* (an audit or proof), never circularly at another undischarged assumption.

3 The seam model

3.1 Standards as interface plus assumption set

Definition 3.1 (Standard, specification, and need). We separate three things the informal literature often blurs.

- $\text{Iface}(X)$ is the *syntactic interface* of a standard X : externally callable selectors with argument and return types and declared events. This is what a compiler checks.
- $\text{Spec}(X)$ is the set of behaviours the standard *explicitly requires* (trace properties or hyperproperties), e.g. “**transfer** moves the stated token and emits **Transfer**.”

- $\text{Need}(C, \rho)$ is the set of behaviours a *particular component* C *relies upon* at a particular seam ρ . A need is a property of the integrator, not of the standard: it may be strictly stronger than $\text{Spec}(X)$, in which case conforming to X does not discharge it.

The crucial move is that the assumptions which cause composition faults are *needs*, not parts of the standard: ERC-20 does not require exact delivery, but a vault may *need* it. Folding such expectations into “the standard’s assumptions” smuggles community folklore into the normative text; keeping **Spec** and **Need** apart is what makes the fault analysis precise. Table 2 classifies, for ERC-20, where each commonly-cited “ERC-20 assumption” actually lives.

Category	Example (ERC-20 ecosystem)
Explicit standard requirement (Spec)	<code>transfer/approve/transferFrom</code> move tokens; <code>Transfer/Approval</code> events emitted
Standard under-specification / ambiguity	return-value convention (revert vs. return <code>false</code>); nonzero→nonzero allowance race
Non-standard implementation behaviour	fee-on-transfer, rebasing, blacklist, pause switch
Integrator need (Need)	exact received amount; absence of transfer callbacks; balance conservation
Required mitigation	balance-delta accounting adapter; allowlist; wrapper; ERC-165 feature-detection

Table 2. The commonly-cited “ERC-20 assumptions” are heterogeneous. Some are explicit requirements, some are under-specification, some are non-standard implementation behaviours, and some are *integrator needs* that the standard never promised. Composition faults arise when a need (row 4) is not met by the chosen implementation (row 3), and are closed by a mitigation (row 5). Callback semantics, often listed as an “ERC-20 hazard,” in fact belong to other standards (the ERC-777 lineage).

3.2 Seams and the two faults of composition

Definition 3.2 (Seam). Let components C_X, C_Y implement standards X, Y and interact. The *seam* $\rho(C_X, C_Y)$ is the pair of directed need sets ($\text{Need}(C_Y, \rho) \upharpoonright X$, $\text{Need}(C_X, \rho) \upharpoonright Y$), where $\text{Need}(C_Y, \rho) \upharpoonright X$ is the subset of C_Y ’s needs whose truth depends on C_X ’s behaviour at this boundary (and symmetrically). A seam is *directed*: each side both offers guarantees and relies on needs.

We now separate two failure phenomena that are routinely conflated and that have *different decidability and different remedies*.

Definition 3.3 (The two faults). Fix a seam, a need $A \in \text{Need}(C_Y, \rho) \upharpoonright X$, and let G_X denote the guarantee C_X offers at the seam.

- **Specification conflict.** The *requirements* are jointly unsatisfiable: $\llbracket A \rrbracket \cap \llbracket G_X \rrbracket = \emptyset$, so *no* implementation could serve both. Example: C_Y needs “transfers are irreversible” while C_X ’s role requires “transfers can be clawed back by an admin.” This is a *design* error detected at the specification level. It is decidable *only after fixing a decidable specification fragment* — e.g. propositional or LTL safety, effectively-propositional (EPR) first-order constraints, or formulas in an SMT-decidable theory under bounded quantification; full first-order satisfiability over the state vocabulary is undecidable, so the fragment must be chosen and stated, not assumed.
- **Seam non-conformance.** The requirements are co-satisfiable, but the *chosen implementation* C_X has runs that leave A : $\text{Tr}(C_X) \not\subseteq \llbracket A \rrbracket$. Example: C_Y needs exact delivery, the

standard permits it, but the *particular token* C_X charges a fee on transfer. This is an *integration* error, and detecting it is the model-checking problem $\text{Tr}(C_X) \subseteq \llbracket A \rrbracket$, *undecidable in general* for EVM contracts (Turing-expressive modulo the gas bound).

Remark 3.1 (The flagship example, correctly located). The canonical failure “a rebasing token under an exact-conservation vault” is *seam non-conformance*, not specification conflict. The property “balances are conserved across transfers” and the property “a vault assumes conservation” are trivially co-satisfiable — any non-rebasing token witnesses both — so their meet is non-empty. The fault is that the *deployed rebasing token’s* traces leave the conservation set. An audit method that tests “ $\llbracket P \rrbracket \cap \llbracket Q \rrbracket \neq \emptyset$ ” would wrongly pass this composition. The correct check is the component-conformance inclusion $\text{Tr}(C_X) \subseteq \llbracket A \rrbracket$, which is the hard, undecidable-in-general check that heavyweight verification exists to attack. Conflating the two makes the easy check look like it covers the hard one; it does not.

Figure 1 renders the distinction. Specification conflict is a relation between two *requirement sets*; seam non-conformance is a relation between an *implementation’s reachable behaviour* and a requirement set.

(a) **Specification conflict** (decidable, rare) (b) **Seam non-conformance** (undecidable, common)



Figure 1. The two faults of composition. (a) Two *requirements* are jointly unsatisfiable — a design error, decidable at the spec level. (b) The requirements are co-satisfiable but a *chosen implementation’s* reachable runs leak outside the assumed property — an integration error, the undecidable model-checking problem that fuzzing and formal verification target. The rebasing-token failure is case (b), not (a).

Principle 1 (Seam primacy, refined). In a composed EVM system the directed seam is the natural unit of the *assurance argument*. For the argument to be complete at a seam, two conditions are necessary: (i) the two sides’ requirements do not conflict, and (ii) each chosen component conforms to every need its neighbour relies on. Condition (i) is a design check, decidable within a chosen specification fragment; condition (ii) is the verification burden, sized by the logical class of the need. These conditions are necessary for a complete seam-level argument; they are *not* a characterisation of system safety, which also depends on intra-component correctness, the completeness of the property catalogue, and the adequacy of the threat model (Section 8).

3.3 Threat model

An undischarged assumption is a latent fault; an adversary is what converts it into a loss. We fix a standard EVM adversary $\mathcal{A}$ with the following capabilities, each of which we will attach to specific seams. The capabilities are deliberately composable: incident data shows that modern exploits are rarely single-vector, instead *chaining* flash liquidity, oracle perturbation, a logic flaw, and a callback or proxy weakness within one atomic transaction [10, 9].

Definition 3.4 (Adversary $\mathcal{A}$). $\mathcal{A}$ is a probabilistic polynomial-time agent that may, within consensus rules:

- T1. Order and interpose.** Observe the mempool and choose the order of transactions within a block, insert its own transactions before/after a victim (sandwiching), and re-enter a victim contract through any callback the victim exposes (reentrancy).
- T2. Borrow unbounded liquidity atomically.** Take a flash loan, so any invariant that holds only “under normal capital constraints” may be probed at arbitrary scale within one transaction.
- T3. Perturb oracle inputs.** Move the price on any venue an oracle reads, within the limits of its own capital amplified by T2, and exploit staleness or push/pull latency windows.
- T4. Capture or front-run governance.** Acquire upgrade or parameter-setting authority (by token accumulation, key compromise, or a mis-scoped role) and, for proxy systems, install new logic; or exploit the window between a governance action’s announcement and its effect.
- T5. Supply adversarial components.** Provide a token, hook, adapter, or module that conforms to the interface but violates needs (the case-(b) fault of Definition 3.3).
- T6. Act across domains.** For systems that span rollups or chains, exploit the message-passing layer: replay or forge a cross-domain message, induce domain confusion, or rely on a finality, latency, or sequencer assumption that does not hold (L1 finality delay, sequencer censorship, forced-inclusion delay, data-availability failure, reorg). Peripheral for a single-chain system, central for any bridged or cross-rollup one.
- T7. Exhaust resources.** Drive a composed path to exceed realistic gas bounds or fail to terminate within them: unbounded loops over positions, hook- or callback-induced gas griefing, storage-bloat denial-of-service, or a liquidation that is correct but unprofitable after gas so that no rational keeper acts. This is the operational form of “Turing-expressive modulo gas”; it is only expressible if gas is in the trace alphabet (Table 1).
- T8. Capture delegation signatures.** Post-Pectra, exploit the EIP-7702 authorization surface directly: phish or replay a delegation signature, or seize an account through an unprotected delegate initialiser — an account-takeover surface distinct from governance capture (T4) and documented in the security literature around 7702 [23].

We do not claim this list is exhaustive of the mid-2026 landscape; further vectors — private-orderflow and builder/searcher fragmentation, solver/intent-layer adversaries, and operational failures (admin key loss, multisig unavailability, emergency-module timelock bypass, ignored monitoring) — sit at the edges of the on-chain model and enter the ledger as **operator-assumption** leaves (Definition 3.5) rather than as formal capabilities here.

3.4 Compositional soundness (safety fragment)

We can now state the result that licenses reasoning about a whole system from its seams. It is a *one-directional soundness* theorem, scoped to the safety fragment, following the Abadi–Lamport discipline (Section 2) [6]: guarantees are safety properties and the dependency must be well-founded. We first fix the typing of the graph, because blending node kinds is what makes informal versions of this argument slippery.

Definition 3.5 (Typed discharge graph). A composed system with seams $\{\rho_i\}$ induces a directed graph $\mathcal{D}$ whose nodes are typed as *needs* (a neighbour’s relied-upon property A), *guarantees* (a property G a component offers, under a component contract “**Need** $\Rightarrow G$ ”), or *leaves*. A leaf is either an *axiom* (a protocol fact, e.g. transaction atomicity) or an *external attestation* of the form $\text{Tr}(C) \subseteq \llbracket G \rrbracket$ for a concrete component C . Edges are entailments oriented *from the stronger*

Seam	Undischarged assumption that becomes exploitable	Capability
token $\rightarrow$ vault	exact delivery / conservation / no callback	T5, T1
oracle $\rightarrow$ market	price freshness and manipulation-resistance	T3, T2
hook $\rightarrow$ pool (Uniswap v4)	hook does not rewrite economics; caller identity	T5, T1
flash-loan callback	receiver returns control in a consistent state	T2, T1
proxy $\rightarrow$ implementation	only authorised upgrades; storage-layout stability	T4
EOA delegation (EIP-7702)	delegate initialiser protected; <code>tx.origin</code> assumptions; delegation-signature authenticity	T4, T1, T8
gas / resource seam	path terminates within realistic gas; liquidation profitable after gas	T7
bridge / messaging seam	message finality, replay protection, domain separation, authenticity, bounded latency	T6

Table 3. Each seam paired with the adversary capability (Definition 3.4) that weaponises its undischarged *need*. The design rules (Section 6) record and close the need; the threat model says what happens if it is left open.

property to the weaker one it discharges: an edge $G \rightarrow A$ exists when $\llbracket G \rrbracket \subseteq \llbracket A \rrbracket$ (“ G entails, hence discharges, the need A ”), and an edge $\ell \rightarrow G$ exists when a leaf ℓ establishes G . With this orientation a topological order runs from leaves through guarantees to the needs they ultimately satisfy, and acyclicity is the well-foundedness condition.

Crucially, a leaf is not a bare assertion but a pair (claim, e) where e is an *evidence class*, because a Certora proof and a documentation sentence both assert $\text{Tr}(C) \subseteq \llbracket G \rrbracket$ yet carry incomparable epistemic weight. As a coarse default ranking — refined in Section 9.4 into a tuple model whose order is only *partial* — we use the classes

$$e \in \{ \text{axiom, machine-proof, bounded-proof, fuzz-campaign, static-analysis,} \\ \text{audit-attestation, economic-assumption, operator-assumption, doc-claim} \},$$

listed in roughly decreasing default strength (an axiom is discharged by the execution model; a doc-claim is mere documentation), with the explicit caveat that this default is only a tie-breaking prior: Section 9.4 shows that scope and coverage can make a broad fuzz campaign outrank a narrow audit, so the true order is partial, not total. A ledger is then *well-formed* as a graph (Definition 3.5 conditions) *and* carries, at every leaf, an evidence class; its *epistemic strength* is bounded by the weakest evidence on any path supporting a critical need. A graph-valid ledger every one of whose solvency leaves is a **doc-claim** is well-formed but epistemically weak — a distinction the framework makes visible rather than hiding, and which Table 11 uses to score defects.

Theorem 3.1 (Assume–guarantee soundness, safety fragment). *Suppose (i) every guarantee node G is a safety property; (ii) $\mathcal{D}$ is acyclic; (iii) every need relied upon by some guarantee has an incoming entailment edge $G \rightarrow A$, every guarantee has its supporting leaves, and every leaf holds (axioms by the execution model, attestations by hypothesis). Then the composed system satisfies the conjunction of all seam guarantees: $\text{Tr}(S) \subseteq \bigcap_i \llbracket G_i \rrbracket$.*

Proof. By (ii) $\mathcal{D}$ admits a topological order from leaves toward needs; by (i) each guarantee is a closed set preserved under prefix-limits, the condition Abadi–Lamport [6] require to exclude vacuous circular discharge. Induct along the order. *Base*: each leaf holds of S — an axiom by the execution model, an attestation $\text{Tr}(C) \subseteq \llbracket G \rrbracket$ by hypothesis. *Step*: take a guarantee G with component contract “**Need**(G) $\Rightarrow G$ ” whose supporting leaves and whose needs $A_1, \dots, A_m$ all precede it. Each A_k is established by an earlier guarantee G_k with $\llbracket G_k \rrbracket \subseteq \llbracket A_k \rrbracket$ (edge $G_k \rightarrow A_k$),

and by the induction hypothesis $\text{Tr}(S) \subseteq \llbracket G_k \rrbracket \subseteq \llbracket A_k \rrbracket$, so $\text{Need}(G)$ holds of S . The component offers G whenever its needs hold; since G is safety, “holds whenever needs held through the previous step” lifts to “holds” without circularity, giving $\text{Tr}(S) \subseteq \llbracket G \rrbracket$. Ranging over all seams, $\text{Tr}(S) \subseteq \bigcap_i \llbracket G_i \rrbracket$. $\square$

Remark 3.2 (Why safety, and where liveness goes). The theorem covers only safety guarantees, because the Abadi–Lamport closure step fails for liveness. Composed systems also rely on *progress* and *economic-liveness* obligations — e.g. “an undercollateralised position is eventually liquidated.” We therefore keep two ledgers: a *safety ledger*, to which Theorem 3.1 applies, and a separate *progress ledger*, whose obligations require fairness assumptions or bounded-horizon reformulation and are *not* discharged by Theorem 3.1. The case study (Section 7) exhibits both, and the formal section (Section 8) keeps the claim about the whole system honest about this split.

Corollary 3.2 (How a safety-ledger argument fails). *If the conclusion of Theorem 3.1 fails to be established, then the safety ledger has at least one of: a missing edge (a relied-upon need with no entailing guarantee), a false leaf (a violated axiom or incorrect attestation), or a cycle (unstratified mutual reliance). The converse does not hold: a well-formed, fully discharged safety ledger does not by itself entail that the deployed system is safe, because safety also depends on obligations outside the ledger — intra-component correctness, the completeness of the chosen property catalogue, the adequacy of the threat model, and the truth of every leaf attestation (Section 8). Corollary 3.2 characterises failure of the argument, not unsafety of the system.*

Corollary 3.2 corrects the tempting but invalid reading “the composition is unsafe iff some need is undischarged.” Undischarged needs are one route to unsafety and the only route the ledger detects; they are neither necessary nor sufficient for unsafety in general, which is why the result is stated one-directionally and relative to the ledger.

3.5 Bounded liveness and economic needs

Most economically critical DeFi obligations are liveness, not safety: *liquidation eventually happens, a withdrawal eventually settles, the oracle eventually updates, a redemption queue eventually drains*. A safety-only theorem leaves these outside the formal result, which would be a serious gap. We recover a large class of them by the standard observation that *bounded* liveness is itself safety.

Definition 3.6 (Bounded-liveness need). A *bounded-liveness need* has the form: under a stated fairness assumption F (e.g. “at least one honest keeper acts each epoch,” “the sequencer includes a forced transaction within its delay bound”), if trigger event ϕ occurs at time t , then response event ψ occurs by $t + k$ for a fixed horizon k measured on a stated clock. We write this $\Box(\phi \Rightarrow \Diamond_{\text{clock}}^{\leq k} \psi \mid F)$, where $\text{clock} \in \{\text{evm-step, transaction, block, oracle-round, l2-batch, governance-epoch, wall-clock}\}$. The clock is part of the need: “liquidated within k ” means materially different things in blocks, oracle rounds, or governance epochs, and a bridge-finality bound (k L1 confirmations) is not interchangeable with a keeper bound (k blocks).

Proposition 3.3 (Bounded liveness is safety, hence admissible in the safety ledger). *For fixed finite k on a fixed clock, the property $\Box(\phi \Rightarrow \Diamond_{\text{clock}}^{\leq k} \psi \mid F)$ is a safety property: any violation is witnessed by a finite prefix in which ϕ occurred and k steps elapsed without ψ . Consequently, once a progress obligation is reformulated with an explicit horizon k and fairness assumption F (the latter recorded as an operator-assumption or economic-assumption leaf, Definition 3.5), it may be placed in the safety ledger and discharged under Theorem 3.1. Genuinely unbounded liveness (“eventually, with no k ”) cannot be so reformulated and remains in the progress ledger, beyond Theorem 3.1 and beyond single-trace tooling (Theorem 8.2).*

Proof. The set of traces satisfying $\Box(\phi \Rightarrow \Diamond_{\text{clock}}^{\leq k} \psi \mid F)$ is closed: its complement is the set of traces with a finite bad prefix (ϕ at t , no ψ through $t+k$), an open set. Closed properties are exactly the safety properties (Theorem 8.1). Placement in the safety ledger and discharge then follow from Theorem 3.1 with the fairness assumption as a leaf. $\square$

This converts many DeFi obligations into checkable safety needs at the cost of naming a horizon and a fairness leaf: *liquidation within k blocks, oracle price no older than k , withdrawal claimable within k , bridge message finalised after k confirmations*. The reformulation is honest about its cost — the fairness leaf is an explicit, typically **operator-** or **economic-assumption** class leaf, exactly the kind the evidence model (Definition 3.5) is designed to surface rather than bury.

Economic needs as a distinct class. Some needs are neither pure safety nor liveness but *quantitative*: “the cost to manipulate the oracle exceeds the value extractable from the manipulation,” $\text{Cost}_{\text{manip}}(\Delta p) > \text{Extractable}(\Delta p)$; “no donation/inflation attack on first deposit”; “share price is monotone non-decreasing absent fees”; “no bad debt under volatility bounded by σ .” These are quantitative trace properties relative to an *economic model* that must itself be part of the trace alphabet (the economic layer of Table 1). We treat them as a separate **economic-need class** whose leaves carry **economic-assumption** evidence and whose discharge is a model-relative inequality, not a code-level invariant; the framework’s role is to make the economic model an explicit, named dependency rather than an unstated belief. Pure trace-property machinery does not validate the economic model; it only propagates its consequences once assumed.

4 The structuring standards, dissected as guarantee/need pairs

We group standards by composition role. For each we give the guarantee it offers, the *need* a careless integrator wrongly attributes to it, and the seam where the gap bites. Factual claims about standardisation stage and library versions are verified against primary sources as of June 2026 (Section 10 notes the cross-checks). These ecosystem-specific statements are time-stamped and *illustrative of the ledger-construction method*; they are not assumptions the formal model depends on, and the method survives unchanged if a protocol version or standard status moves.

4.1 Asset interfaces: what flows across a seam

ERC-20 (Final). Guarantees a uniform transfer/approve surface; does *not* guarantee exact delivery, balance conservation, callback-freedom, or return-value uniformity (Table 2). Every vault, market and AMM inherits these gaps at its deposit seam. *Remedy (R2 below)*: normalise through an adapter and integrate against the adapter, never the raw token.

ERC-2612 (Final) and Permit2. ERC-2612 adds **permit** (an EIP-712 approval signature), removing a transaction at the cost of a *signature seam*: the domain separator must bind (chain id, token), and the nonce model must be understood. For tokens not under one’s control, Uniswap’s *Permit2* supplies a uniform approval semantics over heterogeneous tokens and is the pragmatic compatibility layer.

ERC-4626 (Final) and its tiered successors. Standardises the tokenised single-asset vault. Its load-bearing assumption is *preview faithfulness*: integrators read `previewDeposit`/`previewRedeem` as predictions of the realised exchange. This breaks under asynchrony, caps, or adverse rounding. The successor standards split off exactly the cases that break it: ERC-7540 (asynchronous request/settle flows), which makes the break explicit by *mandating that*

previewDeposit/previewMint (async deposit) and previewRedeem/previewWithdraw (async redeem) MUST revert for all callers [30], so an integrator that relied on preview gets a transaction failure rather than a wrong number; ERC-7575 (externalised `share` token and multiple entry points) [31]; and ERC-7535 (native-asset vaults), which introduces behavioural differences from ERC-4626 — `msg.value`, payable `deposit/mint`, and ETH-specific transfer behaviour that the EIP itself flags as a fragmentation and security concern [32]. A relevant 2026 detail confirming the fragility: OpenZeppelin’s ERC-4626 [20] now computes `maxWithdraw` via `maxRedeem` and `previewRedeem`, so that changes to the preview functions correctly propagate to the max functions — an acknowledgement that preview/limit consistency is a real seam.

Multi-token and NFT interfaces. ERC-1155 (Final) is more gas-efficient than ERC-20/721 when many asset classes coexist. ERC-6909 [27] is a published Standards-Track ERC specifying a deliberately minimal multi-token interface (ERC-165 required, callbacks and batching removed relative to ERC-1155), used in Uniswap v4-style internal accounting; depending on source freshness, some library documentation still describes it as a draft, so we rely on the rendered EIP specification rather than a bare status label. The ERC-721 extension family — ERC-2981 (royalty surface, no economic enforcement), ERC-4906 (metadata-update events), ERC-4907/ERC-5006 (time-limited user role), ERC-4494 (the 2612 permit pattern for NFTs) — each adds capability without altering the core transfer needs.

4.2 Signatures and accounts: who may authorise across a seam

The essential authorisation triplet is EIP-712 + ERC-1271 + ERC-2612: typed messages, contract-signature validation, and token approvals. The decisive design fact is that *the moment any signer may be a contract* (a Safe, a smart account), ERC-1271 is unavoidable, and an integration that special-cases EOA signatures is wrong by omission.

EIP-7702 (Final, live since May 2025). We correct a common framing: EIP-7702 [22] is not “emerging”; it shipped to Ethereum mainnet in the Pectra hard fork [24] on 7 May 2025 as transaction type `0x04` (“set code”), letting an EOA adopt a delegate contract’s code while keeping its address, key, and storage. Two security facts make it a first-class seam, not a footnote. First, under a proxy-style delegate an *unprotected initialiser lets an attacker seize the account* [25] — the classic uninitialised-proxy takeover, now applicable to ordinary user accounts. Second, EIP-7702 *breaks* the historical pattern of using `tx.origin == msg.sender` to detect “called directly by a human,” because a delegated EOA can now run contract logic as the transaction origin; defences against atomic sandwiching that relied on this are void. The threat-model tags for this seam are therefore T4 and T1 (Table 3). ERC-4337 (account abstraction over an alternate mempool) remains complementary: 7702 upgrades the account, 4337 standardises how it talks to bundlers and paymasters; ERC-7579 pursues a modular-account standard layered on top.

4.3 Cross-cutting primitives and architectural EIPs

ERC-3156 standardises flash loans, whose *callback* is the seam to secure (T2/T1). ERC-1167 and ERC-7511 give cheap immutable clones (the latter saving roughly 200 gas at deploy and 5 per call via `PUSH0`). ERC-165 is the interface-detection primitive that lets a seam *feature-detect rather than assume*. A subset of protocol EIPs are *architectural* because they change which compositions are economical: EIP-1153 (transient storage) [26] makes Uniswap v4 flash accounting [33] and Balancer v3 router accounting practical and reshapes reentrancy guards (OpenZeppelin ships `ReentrancyGuardTransient`); EIP-5656 (`MCOPY`) and EIP-3855 (`PUSH0`) drive minimal bytecode;

EIP-2929/3529/2200 (warm/cold access, refund reduction) settle storage-versus-transient trade-offs; EIP-4844 (blobs) governs rollup data-availability cost.

4.4 Libraries as risk postures

The library choice is a choice of risk posture. OpenZeppelin Contracts (verified at v5.6.1, MIT, audited-latest tag) [19, 20] is the stable-API default for standards, access control, upgradeability and cryptographic utilities, and its recent releases push the base layer toward smart accounts and cross-chain messaging (WebAuthn/P256 signers, ERC-7739, ERC-7930 interoperable addresses, the ERC-7786 cross-chain recipient, transient reentrancy guards). Solady and Solmate trade audit-surface for gas, appropriate only for teams that can absorb more internal review. PRBMath is the fixed-point reference (SD59x18, UD60x18). Safe is the mature production smart-account. Permit2 is the universal-approval compatibility primitive; Multicall3 the read/batch aggregator. A governance note relevant to operational risk: OpenZeppelin is sunsetting the hosted Defender platform (final shutdown 1 July 2026) [21] in favour of open-source Relayer/Monitor — teams depending on Defender automation for upgrades or monitoring must migrate, which is itself a change to the governance seam.

5 The DeFi platforms, dissected by extension point

Large protocols are no longer applications but *platforms of primitives*. What distinguishes them is *where the extension point lives* — hook, adapter, router, proxy, vault, oracle, registry, or risk-parameter surface — because that is where composability and risk co-locate. Table 4 summarises; the paragraphs state each platform’s principal seam and the assumption an integrator must not make.

Platform	Principal seam / extension point and its hazard	Capability
Uniswap v4	singleton <code>PoolManager</code> ; <i>hooks</i> can rewrite swap economics; inside a hook <code>msg.sender</code> is the manager, not the user	T5, T1
Aave V3	stable <code>Pool</code> core over a <i>governed parameter surface</i> (caps, pauses, eModes, debt ceilings, oracle config)	T4, T3
Compound III	<code>Comet</code> logic behind a transparent proxy; risk in governance roles, <code>ProxyAdmin</code> , configurator, feeds	T4, T3
Morpho Blue	immutable oracle-agnostic core; sophistication and risk rise into <i>Vaults V2</i> adapters, caps, timelocks, curator permissions	T5, T4
Balancer V3	<code>Vault/Router/Hooks</code> triptych; integrate via router; hooks create a v4-like economic surface	T5, T1
Curve NG	NG pool families + <code>RouterNG</code> + <code>crvUSD/LLAMMA</code> ; only sanctioned NG-pool oracles are adequate for permissionless markets	T3, T5

Table 4. The six structuring platforms, each by its principal seam and the adversary capability (Definition 3.4) that the seam’s assumption guards against.

Uniswap v4 — the singleton and the hook seam. The unit of integration is a single `PoolManager`, not a pool contract [33]; extensibility is pushed to hooks, with flash accounting (enabled by EIP-1153) and ERC-6909 internal balances. The sharp hazard: inside a hook, `msg.sender` is the `PoolManager`. Authentication, fee-discounts and allow-lists must travel via trusted-router patterns or explicitly propagated data — never inferred from the caller. A hook is not “just a callback”; it can change the economics of the swap (T5).

Aave V3 — the parameter-surface seam. A stable `Pool` core exposes documented layers (`aToken`, `VariableDebtToken`, flash loans, ACL, risk parameters) [34]. Correct integration codes against the *parameter surface*: pause flags, supply/borrow caps, siloed borrowing, debt ceilings, liquidation fees, `eModes`, oracle safety — not the happy path. Composability here is real but conditional on governance (T4) and price feeds (T3).

Compound III — the market-as-service seam. `Comet` (logic) behind a transparent proxy, with `CometExt`, `Configurator`, `ProxyAdmin`. The right mental model is a single centralised per-market service, not interchangeable `cTokens`; risk concentrates in governance roles and feeds.

Morpho Blue + Vaults V2 — small core, large perimeter. The market core is simple, immutable, oracle-agnostic (`IOracle.price()`) [35]; sophistication rises into oracles, interest-rate models, and especially Vaults V2 with adapters, caps, timelocks and an adapter registry. Integration is trivial at the market and hard at the *governance of the risk perimeter*: the unit to version and test is the tuple *oracle + adapter + cap logic + curator permissions* (T5/T4).

Balancer V3 — the Vault/Router/Hooks triptych. The `Vault` externalises accounting (transient-storage based); integrate via a router, never via low-level `Vault` calls; hooks create a risk surface analogous to Uniswap v4 (T5/T1).

Curve NG — the validated-component seam. `StableSwap-NG`, `Twocrypto-NG`, `Tricrypto-NG`, `RouterNG`, and `crvUSD/LLAMMA` gradual liquidation. The documented rule: prefer NG components, verify the pool’s asset type, and treat permissionless lending markets as explicit due-diligence objects — their oracles are adequate only when sourced from sanctioned NG-pool families (T3/T5).

6 Guarded design rules for constructing the discharge ledger

We present selection as a set of guarded *design rules*. These are explicitly engineering heuristics, not a formal decision procedure: they have no completeness or optimality guarantee, and they do not select a unique design. What they do is disciplined and checkable — each rule, when applied, either closes a named seam (by supplying a guarantee that entails a neighbour’s need) or records an obligation for the verification stage, so that following the rules *constructs the discharge ledger* of Definition 3.5. Their soundness claim is therefore modest and relative to that ledger: a rule never silently leaves a need unrecorded. We first justify the control order, then give the rules, then the upgradeability sub-procedure with its trade-off lattice. (A genuine decision procedure would additionally require a formal input language, rule-priority and conflict-resolution semantics, a defined output object, and termination and soundness proofs; we do not claim those here.)

6.1 Why upgradeability is decided first

Proposition 6.1 (Proxy choice dominates the seam graph). *The upgradeability pattern determines (i) whether the implementation is part of the mutable trust surface, (ii) the storage-layout constraint that every later component must respect, and (iii) the governance seam through which adversary capability T4 acts. Hence the proxy decision constrains the admissible discharges of every other seam and must precede them.*

The justification is structural: a Diamond’s facet storage discipline, a UUPS implementation’s `_authorizeUpgrade` guard, and a beacon’s single point of fleet control each impose obligations that downstream token/vault/oracle seams must satisfy. Deciding them last would force rework. This is why Figure 3 branches on upgradeability at the root.

6.2 The rules

R1 (Approval layer). Token under your control and standard $\Rightarrow$ ERC-2612. Heterogeneous/third-party tokens $\Rightarrow$ Permit2. Always: bound approvals in amount and time; feature-detect via ERC-165; never assume `nonzero  $\rightarrow$  nonzero approve` is safe.

R2 (Token-behaviour seam). Classify every value-bearing ERC-20 on {reverts vs returns-false}, {callback: yes/no}, {conservation: exact/rebasing/fee-on-transfer}. Anything other than (reverts, no-callback, exact) $\Rightarrow$ wrap behind a normalising adapter; integrate against the adapter. (Discharges the token $\rightarrow$ vault seam against T5.)

R3 (Vault tier). Default ERC-4626; native asset $\Rightarrow$ ERC-7535; asynchronous $\Rightarrow$ ERC-7540, and *do not call `preview*` on the `async` path* — the standard mandates it reverts [30]; multi-asset/external share $\Rightarrow$ ERC-7575. Migration order $4626 \rightarrow 7535 \rightarrow 7540 \rightarrow 7575$ tracks which assumption breaks.

R4 (Authorisation). Build on EIP-712. If *any* signer may be a contract $\Rightarrow$ ERC-1271 mandatory; do not special-case EOAs. Short time-to-market AA $\Rightarrow$ Safe + ERC-4337; add EIP-7702 only with a minimal, initialiser-protected delegate and an explicit review of `tx.origin` assumptions (T4/T1). ERC-2771 only with an explicitly trusted forwarder.

R5 (Credit/treasury at scale). Aave V3, Morpho Blue/Vaults, or Safe per the business layer; treat oracles as a first-class security component (T3), not a service dependency; add relational/invariant proofs (Halmos/Certora) on solvency and authorisation; code against the parameter surface (T4).

R6 (Custom DEX/router/executor). Uniswap v4 if differentiation is in hook logic, custom accounting or dynamic fees; Balancer V3 if it is in a mutualised Vault, specialised router, or broad pool/invariant. Both: no third-party hook without a dedicated threat model; never read user identity from `msg.sender` in a hook.

R7 (Upgradeability). See Section 6.3.

R8 (Generalist default). OpenZeppelin for standards; UUPS only with a real upgrade need and a real timelock; ERC-2612 when mastered else Permit2; Foundry + Slither + Echidna standing; fork-test every material integration.

Proposition 6.2 (Conflict-awareness of the rule set). *The rules are not jointly exhaustive of design space, and two upgradeability conditions can co-fire (e.g. a homogeneous fleet that also wants facet modularity). The procedure resolves such co-firing by Proposition 6.1’s precedence order (fleet/Beacon before facet/Diamond) and flags the residual as a specification-conflict candidate (Definition 3.3(a)) to be checked at the specification level before implementation. The procedure therefore never silently commits a conflicting pair.*

6.3 Upgradeability: where the logic of change lives

The question is not “which proxy” but *where the governance of change lives*. Figure 2 renders the trade-off as a partial order on two axes (gas cost vs. structural risk concentration); Table 5 states the patterns; R7 selects.

Pattern	Std	Where upgrade logic lives / chief risk (adversary T4)	Gas
Transparent	1967	in the proxy; explicit admin/user split; governance-error risk	High
UUPS	1822+1967	in the implementation; missing <code>_authorizeUpgrade</code> or non-UUPS impl bricks upgrade; <i>shared 1967 slot</i> makes UUPS-behind-Transparent dangerous	Low
Beacon	1967+beacon	in the beacon; atomic fleet upgrade; systemic risk concentrated on the beacon	Medium
Diamond	2535	facet mapping; near-unlimited modularity; heavy storage/tooling/cut-governance complexity	Variable
Clones	1167/7511	no upgrade logic; immutable; fixes only by migration	Very low

Table 5. Upgradeability patterns. OpenZeppelin prefers UUPS to Transparent for gas and flexibility but documents a critical hazard: because both share the ERC-1967 implementation slot, a UUPS implementation behind a `TransparentUpgradeableProxy` can expose upgrade capability to non-admins. The proxy pattern is part of the security interface, not a deployment detail.

R7. Single contract or shallow system needing upgrade, with a serious timelock $\Rightarrow$ UUPS. Crystal-clear admin/user separation $\Rightarrow$ Transparent. Homogeneous fleet to patch atomically $\Rightarrow$ Beacon. Immutable instances (simple vaults, modules, escrows) $\Rightarrow$ clones. Genuine facet modularity or contract-size pressure *and* above-average review/observability $\Rightarrow$ Diamond; otherwise not. *Never place a UUPS implementation behind a Transparent proxy.*

6.4 Migration paths

Approvals: `approve` $\rightarrow$ ERC-2612 (own token) $\rightarrow$ Permit2 (heterogeneous). Wallets: EOA $\rightarrow$ Safe/4337; EIP-7702 now *available today* (post-Pectra) as an in-place upgrade, with the initialiser and `tx.origin` caveats of R4. Vaults: as R3. Proxies: Transparent $\rightarrow$ UUPS to lighten cost; fleets $\rightarrow$ Beacon. Protocol: Uniswap v3 $\rightarrow$ v4 and Morpho Vaults V1 $\rightarrow$ V2 require an explicit adaptation layer because the *unit of integration* changes.

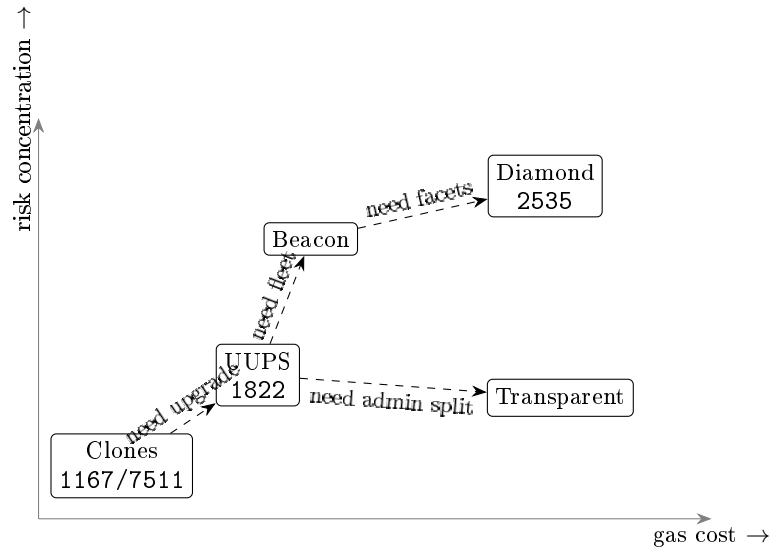


Figure 2. Upgradeability patterns ordered by gas cost (horizontal) and structural risk concentration (vertical). Dashed arrows read “move here only when the stated need arises.” The default (R7/R8) is the low-cost, low-concentration corner; each step up or right must be justified by a concrete requirement, because each adds either cost or a concentrated T4 target.

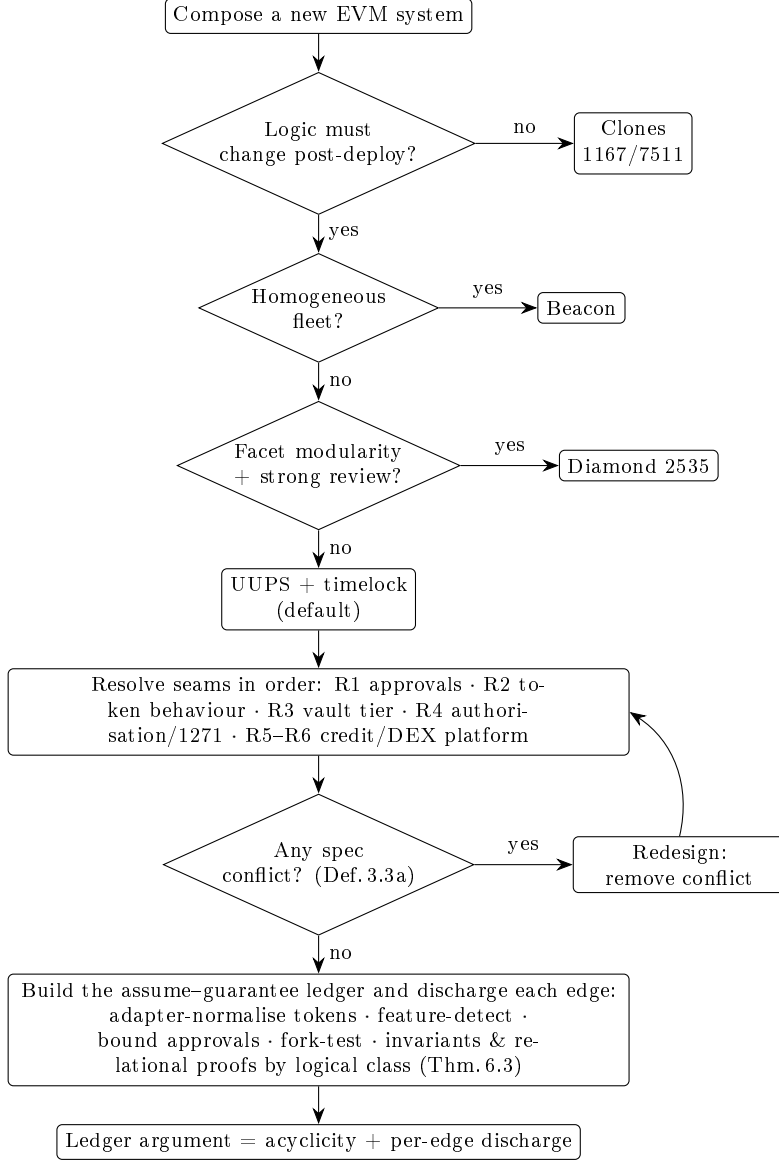


Figure 3. A guarded design workflow for constructing the discharge ledger. Upgradeability is decided at the root (Proposition 6.1); seams are then resolved by R1–R6; a specification-conflict gate (Definition 3.3(a)) catches design errors *before* implementation; the workflow terminates by constructing the ledger, whose two checkable conditions are acyclicity (decidable) and per-edge discharge (delegated to the tool dictated by each obligation’s logical class). This is a workflow, not a decision procedure: it has no completeness or uniqueness guarantee.

7 Worked case study: a yield router, end to end

We carry one realistic composition through the whole method to show that the calculus does work, and that it surfaces a genuine fault. The system: a **yield router** that accepts a user deposit of a stablecoin, wraps it in an **ERC-4626** vault, supplies the vault assets into **Aave V3** to earn yield, prices the position with a **Chainlink** feed for a borrow facility, and is deployed behind a **UUPS** proxy under a timelocked governance. This is an ordinary DeFi pattern; we show it has a non-obvious composition fault.

7.1 Step 1 — enumerate components and standards

C_{tok} : the deposited stablecoin (ERC-20). C_{vault} : the **ERC-4626** wrapper. C_{mkt} : Aave V3 Pool. C_{orac} : Chainlink feed (push oracle). C_{proxy} : UUPS (ERC-1967+1822). The guarded workflow (Section 6) selects UUPS at the root (single shallow system, real upgrade need, timelock present), then R2 (token), R3 (vault), R5 (credit) at the seams.

7.2 Step 2 — enumerate properties and seams

The relevant guarantee/need pairs:

- P_1 (*observable received amount*, safety): the vault credits the amount it actually receives, i.e. a standard transfer preserves total balances and delivers the nominal amount. *Offered* by a standard stablecoin; *needed* by C_{vault} at the deposit seam. (The constrained adapter below discharges only the observed-delta form P'_1 ; see Section 7.)
- P_2 (*preview faithfulness*, safety): `previewDeposit` predicts realised shares. *Offered* by a synchronous uncapped vault; *needed* by the router’s accounting. (Note: against an **ERC-7540** async vault this need is not merely unmet but actively reverts [30], so the seam fails loudly rather than silently — the safer of the two failure modes.)
- P_3 (*price freshness & manipulation-resistance*, safety): the oracle price is recent and not cheaply movable. *Offered* by a correctly configured Chainlink feed with a staleness check; *assumed* by the borrow facility’s solvency logic.
- P_4 (*solvency*, safety): vault assets $\geq$ liabilities at every reachable state. *Offered* by the router; depends on P_2, P_3 .
- P_5 (*authorised-upgrade-only*, safety): only the timelock can change implementation. *Offered* by the UUPS `_authorizeUpgrade` guard; depends on storage-layout stability P_6 .
- P_6 (*storage-layout stability*, safety): upgrades preserve the storage layout. *Offered* by disciplined storage and OZ upgrade checks.
- P_7 (*liquidation liveness*, liveness): an undercollateralised position is eventually liquidated. *Offered* by keepers/incentives; *not* a safety property.

Seams: $\rho_1 = \text{token} \rightarrow \text{vault}$ (needs P_1); $\rho_2 = \text{vault} \rightarrow \text{router}$ (needs P_2); $\rho_3 = \text{oracle} \rightarrow \text{router}$ (needs P_3); $\rho_4 = \text{proxy} \rightarrow \text{logic}$ (needs P_5, P_6).

7.3 Step 3 — build the discharge graph and find the fault

Figure 4 is the discharge graph (Definition 3.5). Reading it bottom-up: P_4 (solvency) relies on P_2 and P_3 ; P_2 relies on the vault being synchronous and uncapped; P_3 relies on a staleness guard and on the feed not being cheaply movable; P_5 relies on P_6 . The *leaves* are external attestations

(the Chainlink feed’s operational guarantees; an audit of the vault) and axioms (transaction atomicity).

Two faults surface mechanically:

1. **A missing edge at ρ_1 (seam non-conformance, case (b)).** If the deposited stablecoin is in fact rebasing or fee-on-transfer, $\text{Tr}(C_{\text{tok}}) \not\subseteq \llbracket P_1 \rrbracket$. No guarantee discharges the need P_1 ; by Corollary 3.2 the *safety-ledger argument fails* at this seam (and the concrete hazard is real: the vault’s share accounting drifts from its asset holdings, and under T2 a flash-funded deposit/withdraw cycle makes the drift extractable). *Remedy:* R2 — interpose a normalising adapter that re-establishes P_1 as a *measured* guarantee by accounting the observed balance delta rather than the nominal amount, closing the edge.
2. **A latent cycle through liveness at ρ_3/P_7 .** The borrow facility’s solvency P_4 tacitly relies on *timely* liquidation P_7 , but P_7 is liveness and lives in the separate progress ledger (Remark 3.2), not the safety ledger Theorem 3.1 covers; worse, if liquidation incentives are paid from the very buffer whose solvency is in question, the graph acquires a cycle $P_4 \rightarrow P_7 \rightarrow P_4$, violating Theorem 3.1(ii). *Remedy:* stratify — fund incentives from an independent reserve so P_7 ’s discharge no longer depends on P_4 , restoring acyclicity — and move P_7 to the liveness tool-track, where it needs a fairness assumption or a bounded-horizon reformulation (it is not establishable by fuzzing, Theorem 8.2).

The normalising adapter of the first remedy supplies a *constrained* guarantee, not a universal one. It measures the realised transfer and mints shares against that, under a reentrancy guard:

```

1 // Adapter guarantee P1' (CONSTRAINED -- see residual assumptions below):
2 // for tokens whose balanceOf is honest over the call interval and whose
3 // transfer does not reenter the mint path, shares are minted against the
4 // observed local balance delta, not the nominal requested amount.
5 function depositNormalized(IERC20 asset, uint256 amount)
6     external nonReentrant returns (uint256 shares)
7 {
8     uint256 before = asset.balanceOf(address(this));
9     asset.safeTransferFrom(msg.sender, address(this), amount); // SafeERC20: no/false return handled
10    uint256 received = asset.balanceOf(address(this)) - before; // realised, not nominal
11    require(received > 0, "no assets received");
12    shares = _mintSharesAgainst(received); // virtual-offset share math
13 }

```

The discharged property is P'_1 , and naming it honestly matters: the adapter does *not* “make every adversarial token safe.” It changes the accounting from nominal to observed-delta, which closes the fee-on-transfer/rebasing edge *only under explicit residual assumptions* — themselves recorded as leaves in the ledger (Definition 3.5):

```

1 guarantee: P1' AdapterObservedDeltaAccounting
2 residual_assumptions: # each becomes a ledger leaf
3 - balanceOf honest over the [before, after] interval # else delta is forged
4 - transfer does not reenter the mint/redeem path # nonReentrant + check-effects
5 - asset decimals fixed for the vault's lifetime # decimals normalised at config
6 - withdrawal side delta-accounted symmetrically # fee-on-withdraw not covered here
7 - token not blacklist/pause-bricked after deposit # else funds strand; wrapper must handle
8 - first-deposit inflation handled by virtual-offset math # P1' alone does NOT defend this

```

The last residual assumption is the bridge to the attack-pattern reconstruction below: observed-delta accounting and inflation-attack resistance are *different* needs, and conflating them is exactly the kind of seam error the calculus is meant to expose. A fuzzer discharges P'_1 by asserting, over the constrained token class, that `totalAssets` never diverges from the summed `received` deltas; the residual leaves must be discharged *separately*.

7.4 Attack-pattern reconstruction: the ERC-4626 inflation attack as a seam fault

To show the method reconstructs a documented *attack pattern* rather than only a textbook one, we map the well-known ERC-4626 first-depositor inflation (“donation”) attack into the ledger. (We call this an attack-pattern reconstruction, not a named-incident reconstruction: it captures a class of exploit documented by OpenZeppelin and others, not one dated historical event with a specific victim and loss.) The pattern: a vault with no inflation defence lets the first depositor mint one share, then *donate* assets by direct transfer — bypassing *deposit* — so a later depositor’s *convertToShares* rounds to zero and forfeits funds [28, 29].

In seam terms, this is *not* the same need as P'_1 . It is a distinct accounting-seam need, and it must be stated as a *bounded value-loss* property, not merely the visible symptom that a deposit mints a nonzero number of shares (a vault could mint one dust share and still be catastrophically lossy). Fixing a tolerated rounding bound ϵ , the property is

P_8 (*inflation resistance*, safety): for any deposit $a > 0$, the depositor’s received shares are worth at least $(1 - \epsilon) a$

under the vault’s exchange-rate function; equivalently, no prior direct donation or first-depositor manipulation can make a deposit lossy beyond the rounding bound ϵ . (The weak reading “mints > 0 shares” is necessary but far from sufficient: it admits exactly the dust-share rounding the donation attack exploits.) P_8 relies on the vault’s share math, not on the token’s transfer semantics, so the observed-delta adapter P'_1 leaves it entirely undischarged — a *missing edge* even after the fee-on-transfer edge is closed. The documented mitigations (OpenZeppelin’s virtual-offset shares, which bound ϵ directly; dead-share burning; a protocol-funded initial deposit, as Morpho does) are precisely the guarantees that discharge P_8 [28]. The lesson the ledger encodes mechanically: closing one accounting-seam edge (P'_1) does not close another (P_8); each is a separately-evidenced leaf.

Adversarial traces. The two faults and the reconstruction are made concrete by explicit adversarial executions over the layers of Table 1 (economic layer for the flash liquidity, EVM-local for the balances):

1	Trace A -- fee-on-transfer extraction (closed by P'_1):	
2	t0 attacker flash-borrows asset	t1 deposits nominal N
3	t2 token delivers only N-fee	t3 vault mints shares as if N arrived
4	t4 attacker withdraws against inflated shares	
5	t5 repays flash loan	t6 residual vault assets < liabilities
6		
7	Trace B -- inflation/donation attack (closed by P_8 , NOT by P'_1):	
8	t0 attacker is first depositor, mints 1 share for 1 wei	
9	t1 attacker DONATES 1e5 assets by direct transfer (bypasses deposit)	
10	t2 victim deposits V; convertToShares rounds $V \cdot 1 / (1 + 1e5) \rightarrow 0$ shares	
11	t3 attacker redeems the single share, now backed by $1e5 + V$	
12		
13	Trace C -- solvency/liveness cycle (the cycle in D, see Fig. 4):	
14	t0 collateral price drops	t1 position undercollateralised
15	t2 liquidation incentive paid from solvency buffer	
16	t3 buffer impaired	t4 rational keepers do not act
17	t5 solvency worsens	t6 P4 depends on P7, P7 depends on P4

Trace A is the textbook fault; Trace B is the documented incident the adapter does *not* fix (which is the point); Trace C is the circular-assurance fault that informal review misses. The calculus catches A as a missing edge that P'_1 closes, B as a *separate* missing edge requiring P_8 , and C as a cycle in $\mathcal{D}$.

Table 6 is the complete ledger for the case study in tabular form — the artifact an auditor would actually hand off. It makes the whole construction legible at a glance: every need, its

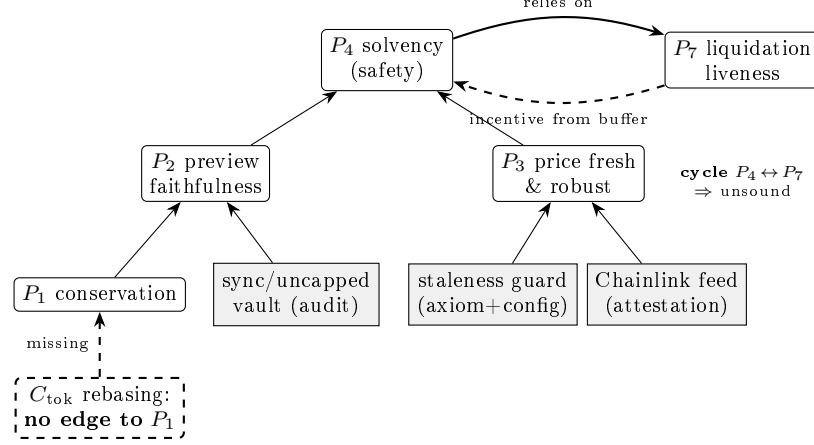


Figure 4. Discharge graph for the yield router (Definition 3.5). Solid edges are discharges that terminate at leaves (grey: axioms/attestations). Two faults appear mechanically: the dashed *missing* edge from a rebasing C_{tok} to P_1 (seam non-conformance, Definition 3.3(b)), and the *cycle* $P_4 \leftrightarrow P_7$ created when liquidation incentives are funded from the buffer whose solvency they are meant to support (a violation of Theorem 3.1(ii)). Remedies: a normalising adapter closes the missing edge; an independent reserve stratifies the cycle.

provider, the discharging guarantee, the evidence and its class, and the status. The two *weak* or *pending* rows (P_4 solvency, P_7 bounded liquidation, P_8 inflation) are exactly where the severity scoring of Table 11 directs attention.

Need	Provider	Guarantee	Evidence class	Status
P'_1 observed-delta accounting	Adapter	ObservedDeltaAccounting	fuzz-campaign	discharged
P_2 preview faithfulness	Vault	sync, uncapped 4626	audit + fuzz	discharged
P_3 oracle freshness	OracleAdapter	staleness bound	static + config	discharged
P_4 solvency	Router	assets $\geq$ liabilities	machine-proof (Certora)	pending
P_5 authorised-upgrade-only	UUPS	timelock-only upgrade	static + formal	discharged
P_6 storage-layout stability	upgrade pipeline	OZ storage diff	static (CI)	discharged
P_7 liquidation within k	Keepers	bounded response F	economic + operator	weak
P_8 inflation resistance	Vault math	virtual-offset shares	audit + fuzz	pending

Table 6. The case-study ledger as an audit artifact. Each row is a serialised ledger node (Section 9.3); the *Evidence class* column carries the coarse evidence class of Definition 3.5, refined by the tuple model of Section 9.4, so a reader sees immediately that P_7 rests on an operator/economic assumption (weak) and that P_4, P_8 are not yet discharged. The ledger’s epistemic strength is the weakest class on a critical path — here the P_7 fairness leaf.

8 Formalisation: the assumption ledger as a mathematical object

We now connect the seam calculus to a companion property catalogue and state a one-directional, catalogue-relative soundness result, being careful about the decidability boundary and about what the result does *not* establish.

8.1 The property of a standard

Definition 8.1 (Catalogued property). Each need $A \in \mathbf{Need}(C, \rho)$ (or standard requirement in $\mathbf{Spec}(X)$) is promoted to a catalogue entry, in the sense of the companion Blockchain Property Ontology [36], $P = (\iota_P, \llbracket P \rrbracket, \chi_P, \rho_P, \alpha_P)$, where ι_P is an opaque identifier; $\llbracket P \rrbracket \subseteq \Sigma$ (trace property) or $\llbracket P \rrbracket \subseteq \mathcal{P}(\Sigma)$ (hyperproperty) is the semantics; $\chi_P = (m, d, D, \ell, v)$ classifies P on five orthogonal axes — modal class m (safety / liveness / k -safety / non-interference), deontic role d , domain D , abstraction level ℓ , and admissible verification modality v ; ρ_P records whether P is *offered* (guarantee) or *relied upon* (assumption) at a given seam; and α_P is the assumption ledger of Definition 3.5. Conformance is $\text{Tr}(S) \subseteq \llbracket P \rrbracket$ for trace properties and $\text{Tr}(S) \in \llbracket P \rrbracket$ for hyperproperties.

This is the object the agent-ontology framework calls a domain constraint, and the form in which the Blockchain Property Ontology [36] catalogues it: each property P to be required (equivalently, each forbidden pattern $\Sigma \setminus \llbracket P \rrbracket$ to be excluded) is promoted to a first-class, classified, independently verifiable entry. The five-axis classification is what lets the verification stage pick the right tool, via the following gate.

8.2 The three structural theorems, stated precisely

Theorem 8.1 (Safety/liveness decomposition; Alpern–Schneider [1]). *In the Cantor topology on $\Sigma = A^{\leq \omega}$, every trace property Π satisfies $\Pi = \text{cl}(\Pi) \cap (\Pi \cup (\Sigma \setminus \text{cl}(\Pi)))$, exhibiting Π as the intersection of a closed (safety) property $\text{cl}(\Pi)$ and a dense (liveness) property. Safety properties are exactly the closed sets; liveness exactly the dense sets.*

Theorem 8.2 (Verification-modality gate). *Let a method be single-trace if it inspects one finite execution prefix at a time (testing, single-run fuzzing, runtime monitoring). Then: (i) for an unbounded liveness property, no single-trace method is complete (no finite observation establishes it); (ii) for a k -safety hyperproperty with $k \geq 2$ (e.g. non-interference, order-independence), no single-trace method is sound-and-complete, since a violation is only witnessed by a k -tuple of prefixes that a single-trace method never assembles. Consequently the modality axis v is constrained by the modal axis m : $m = \text{liveness}$ or $m = k\text{-safety}$ forces v to include a relational or temporal method (bounded model checking for bounded liveness; relational/product-program reasoning, symbolic or deductive, for k -safety).*

Remark 8.1 (Scope of Theorem 8.2). The result is the standard reading of Alpern–Schneider (liveness) [1] and Clarkson–Schneider (k -safety) [3] specialised to single-trace observers; we state it for unbounded liveness because *bounded* liveness (“within k steps”) is itself a safety property and *is* single-trace checkable. The practical upshot fixes the tool map of Table 13.

Theorem 3.1 (assume–guarantee soundness) was stated and proved in Section 3. Together the three theorems give the audit its shape: Theorem 8.1 says every obligation splits into a safety part and a liveness part; Theorem 8.2 says the part’s logical class dictates the admissible tool; Theorem 3.1 says the whole is the conjunction of the parts *provided* the discharge graph is acyclic and bottoms out at holding leaves.

8.3 Soundness of the ledger argument, relative to a catalogue

Theorem 8.3 (Soundness of the safety-ledger argument, relative to a catalogue). *Fix a property catalogue Φ and a threat model $\mathcal{A}$. Let S be a system carrying a typed discharge graph $\mathcal{D}$ (Definition 3.5) over Φ . If*

- (A) **Ledger well-formedness** — $\mathcal{D}$ is acyclic and every Φ -need relied upon by a guarantee has an incoming entailment edge terminating at a leaf (a property of the finite graph, decidable in time linear in $|\mathcal{D}|$ by topological sort and reachability); and
- (B) **Per-edge discharge** — for each edge $G \rightarrow A$ the entailment $\llbracket G \rrbracket \subseteq \llbracket A \rrbracket$ holds (a specification-level check, decidable only within the chosen fragment of Definition 3.3(a)), and for each leaf attestation the inclusion $\text{Tr}(C) \subseteq \llbracket G \rrbracket$ holds (the model-checking problem, undecidable in general, discharged by the tool dictated by Theorem 8.2);

then S satisfies the conjunction of the catalogued safety guarantees: $\text{Tr}(S) \subseteq \bigcap_i \llbracket G_i \rrbracket$.

Proof. (A) and (B) supply exactly the hypotheses of Theorem 3.1 (acyclicity; every leaf holds; every edge a genuine entailment), whose conclusion is $\text{Tr}(S) \subseteq \bigcap_i \llbracket G_i \rrbracket$. $\square$

This is a *one-directional* statement, and deliberately so. The converse — “if S is safe then (A) and (B) hold” — is false, which is why we do not state an **iff**.

Remark 8.2 (What Theorem 8.3 does *not* prove). Four gaps are outside the theorem and must not be read into it. (1) *Catalogue incompleteness*: S may satisfy every catalogued guarantee yet be unsafe through a property no one wrote down. The conclusion is relative to Φ , not absolute. (2) *Ledger incompleteness is not unsafety*: a missing edge means the *argument* is incomplete, not that the deployed system is unsafe — S may be safe by an unmodelled invariant or by luck. Failure of (A) condemns the proof, not necessarily the system. (3) *Attestation fallibility*: each leaf $\text{Tr}(C) \subseteq \llbracket G \rrbracket$ is only as true as its issuer; a false attestation yields a valid-looking ledger over a false premise. (4) *Threat-model adequacy and liveness*: the theorem covers safety guarantees under $\mathcal{A}$; an attacker outside $\mathcal{A}$, or a progress obligation in the separate liveness ledger (Remark 3.2), is not addressed. The value of the framework is that all four gaps become *named*: an undischarged need, an uncatalogued property, a specific attestation, an out-of-model capability. None is hidden, but none is dissolved.

Corollary 8.4 (What is and is not mechanical). *Condition (A) — the bookkeeping — is fully mechanical and cheap. Condition (B) — the content, especially the leaf attestations — is in general undecidable and is the irreducible work of verification, sized per obligation by its logical class (Theorem 8.2). The formalisation does not make the undecidable decidable; it separates the cheap decidable part from the expensive part and localises the latter as named leaves of $\mathcal{D}$.*

We therefore make a narrower claim than “audits become mathematical.” What becomes a mathematical object is the *assumption ledger*, which is the audit artifact: a typed, finite graph whose well-formedness is decidable and whose leaves carry an explicit, classified, tool-matched verification and trust burden. The audit itself remains socio-technical — it includes judgement about property adequacy and threat-model completeness, code comprehension, economic and operational review, and adversarial creativity, none of which the ledger replaces. The honest slogan is: *the assumption ledger is mathematical; the audit remains socio-technical.*

8.4 The bridge to the verifiable-agent framework

Theorem 8.5 (Instantiation). *A composed EVM system S with chosen standards $\{X_j\}$ and discharge graph $\mathcal{D}$ instantiates a verifiable agent whose domain ontology $O = (T_O, \sqsubseteq_O, \Phi_O)$ has type order $\sqsubseteq_O$ given by the domain/level axes of the catalogued properties and a required-property set $\Phi_O = \{P_1, \dots, P_n\}$ — the catalogued safety guarantees the system must satisfy (we state Φ_O as the properties to hold, not their negations, to avoid ambiguity; “forbidden pattern” is then simply the complement $\Sigma \setminus \llbracket P_i \rrbracket$). Conformance is $S \Vdash O \iff \text{Tr}(S) \subseteq \bigcap_i \llbracket P_i \rrbracket$. Under this reading: the system graph (Figure 5) is the dependency graph over which*

Φ_O is enforced; specification conflict (Definition 3.3(a)) is the ontology relation $\llbracket P \rrbracket \cap \llbracket Q \rrbracket = \emptyset$ (*conflictsWith*); seam discharge is assume–guarantee composition in the sense of Theorem 3.1; and seam non-conformance (Definition 3.3(b)) is a component failing the per-edge attestation of Theorem 8.3(B). We state the correspondence only at this set-theoretic level; the companion framework’s categorical apparatus (“core-guarded gluing,” typed pushouts) is not used here and we make no pushout claim — relating seam discharge to a universal-property gluing is deferred to future work (Section 11), since stating it responsibly requires defining objects, morphisms, the guard, and the universal property, which is out of scope.

The restriction is deliberate. There is a defensible *topological* reading that we do use (it is what earns the term “calculus”): trace properties are sets in the Cantor topology on Σ (Theorem 8.1), refinement is inclusion, safety is closure, the liveness residue is the part invisible to any finite-prefix audit, and a seam is locally discharged when the local property spaces agree on their overlap under the stated trace layer and threat model. That reading is honest because it rests only on Theorem 8.1. The stronger categorical reading — composition as a pushout, seams as gluing maps with a universal property — is suggestive but unearned at present, so we keep it as future work rather than decoration.

Engineering object	Catalogue object	Agent-framework object
standard X	property set $\{P\}$ over $\text{Spec}(X)$	required properties $P \in \Phi_O$
need $A \in \text{Need}(C, \rho)$	ledger entry in α_P	relied-upon precondition
guarantee at a seam	$\rho_P = \text{offered}$	offered capability
specification conflict (Def. 3.3a)	conflictsWith : $\llbracket P \rrbracket \cap \llbracket Q \rrbracket = \emptyset$	ill-typed gluing
seam non-conformance (Def. 3.3b)	failed attestation $\text{Tr}(C) \not\subseteq \llbracket A \rrbracket$	component breaks contract
discharge graph $\mathcal{D}$	assume–guarantee ledger	compositional-safety proof tree
audit cond. (A)	ledger well-formedness	well-founded dependency
audit cond. (B)	per-edge / leaf verification	opacity localised in δ_a

Table 7. The three views as one structure, with the two faults kept distinct (the correction of Remark 3.1). Specification conflict is *conflictsWith*; seam non-conformance is a failed leaf attestation. They populate different parts of $\mathcal{D}$ and are checked by different means.

8.5 The composition stack as system dependency graph

Figure 5 renders the canonical stack referenced by Theorem 8.5: authorisation at the entry, routers/hooks/adapters in the middle, markets and vaults at the core, oracles feeding the core, and governance/timelock/proxies as the change layer. The dashed inter-layer boundaries are the seams; the figure is the dependency graph over which Φ_O is enforced.

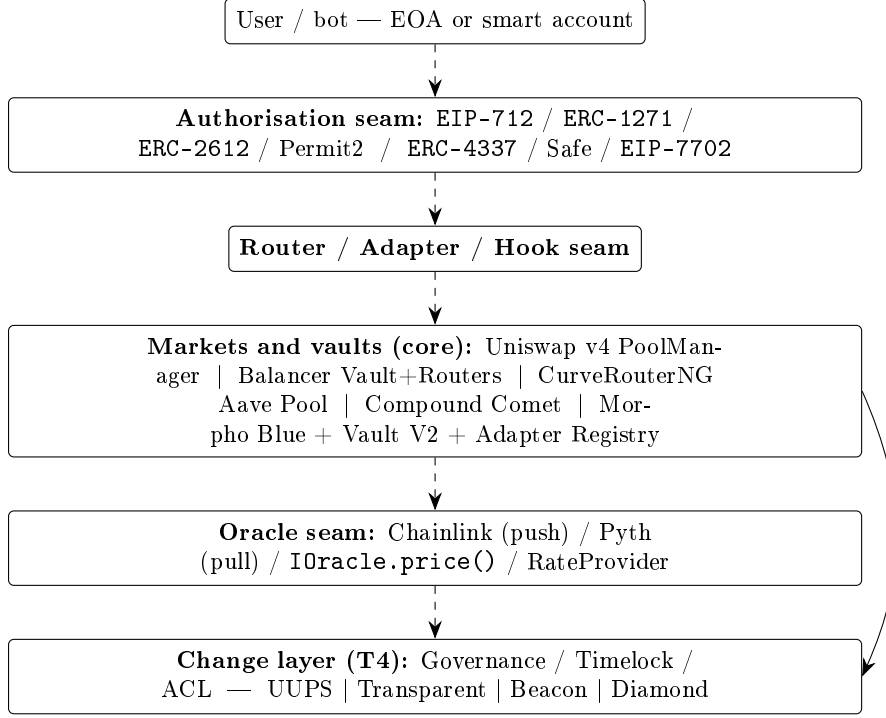


Figure 5. The composition stack of mid-2026 as a system dependency graph (Theorem 8.5). Dashed boundaries are seams; each is tagged in Table 3 with the adversary capability it guards against. Risk concentrates on the boundaries, not the boxes (Principle 1).

9 Making the ledger operational

The preceding sections define the ledger as a mathematical object. This section supplies what turns it from a typed worksheet into an audit-usable artifact: the two-fault spine as a table, a taxonomy of seam types, a concrete property-catalogue fragment with a machine-readable schema, a severity scoring for ledger defects, an explicit statement of what the calculus *cannot* detect, and a comparison to existing audit methodologies.

9.1 The two faults, as a spine

Everything operational rests on the distinction of Definition 3.3, so we restate it as a single table that an auditor can hold in view while building the ledger. The two faults are not variants of one phenomenon: they differ in mathematical shape, in decidability, and in remedy, and the entire method depends on never letting the cheap check (specification conflict) masquerade as coverage of the expensive one (seam non-conformance).

Fault	Math. shape	Example	Detection / remedy
Specification conflict	$\llbracket A \rrbracket \cap \llbracket G \rrbracket = \emptyset$	irreversible transfer vs. admin clawback	SMT in chosen fragment; redesign
Seam non-conformance	$\text{Tr}(C) \not\subseteq \llbracket A \rrbracket$	fee-on-transfer token under exact-delivery vault	verification / fuzzing / audit; adapter, wrapper, or reject

Table 8. The conceptual spine (Definition 3.3). The two faults differ in mathematical shape, decidability, and remedy; conflating them (Remark 3.1) makes the cheap spec-level check appear to cover the expensive implementation-level one.

9.2 A taxonomy of seam types

“Seam” is broad by design, but not everything is a seam: an obligation entirely internal to one component (an arithmetic invariant with no boundary) is *not* a seam need, and belongs to ordinary single-contract verification. The seams that do arise fall into a small taxonomy (Table 9), each with a dominant failure mode and a characteristic evidence requirement.

Seam type	Dominant failure mode	Characteristic evidence
ABI seam (ERC-20→vault)	semantic under-specification	adapter invariant (fuzz)
callback seam (ERC-777, hooks, flash loans)	reentrancy, identity confusion	call-graph + reentrancy proof
accounting seam (vault↔router)	rounding, preview mismatch	conservation invariant
oracle seam (feed→market)	stale / manipulated price	oracle model + manipulation-cost bound
authority seam (governance/proxy)	capture, wrong admin	role invariant + timelock proof
upgrade seam (proxy→impl)	storage collision, auth failure	storage-layout diff + upgrade invariant
cross-domain seam (bridge→receiver)	replay, finality, domain confusion	message-authenticity proof
operational seam (keepers, RPC, frontends)	liveness, censorship, injection	monitoring / SLA / economic model

Table 9. Seam taxonomy. The framework is, at one level, a dependency graph with typed assume-guarantee labels — we say so plainly; its value is the typing, the evidence classes, and the verification-modality assignment, not a claim that “seam” is a novel primitive.

9.3 A canonical property, concretely

To show the catalogue is more than a taxonomy, here is one property fully instantiated, in a machine-readable form that an auditing pipeline can consume. The semantics is a state-trace predicate over the layers of Table 1; the evidence class and threat tags are first-class fields.

```

1  property:
2    id: P1
3    name: ExactReceivedAmount
4    modal_class: safety
5    trace_layer: evm_local
6    seam_type: abi_seam
7    formalization:           # over the evm-local layer
8      expr: "bal_after(vault) - bal_before(vault) == received"
9    refines: BalanceDeltaAccounting
10   conflicts_with:
11     property: NominalAmountAccounting
12     when: "token.behavior in {fee_on_transfer, rebase}"
13   verification_modality: [invariant_fuzzing, symbolic_execution]
14   threat_tags: [T5, T1, T2]
15   discharged_by:
16     guarantee: AdapterObservedDeltaAccounting
17     evidence:
18       class: fuzz_campaign           # see evidence ordering, Def 3.5
19       tool: Foundry
20       artifact: test/AdapterInvariant.t.sol
21       status: passing

```

The schema generalises: each ledger node serialises to such a record, an edge $G \rightarrow A$ is a `discharged_by` link, and ledger well-formedness (acyclicity, every need discharged, evidence

present) is a linear-time check over the serialised graph. This is the precise sense in which the *ledger* — not the audit — is a mathematical object: it is a finite typed graph with a decidable well-formedness predicate and an evidence-class annotation on every leaf.

9.4 Evidence as a tuple, and why the class order is only a partial order

The class ordering of Definition 3.5 ($\text{axiom} \succ \dots \succ \text{doc-claim}$) is a useful default but *not* a defensible total order: a broad, reproducible fuzz campaign over the exact adversarial token class can be stronger evidence than a narrow audit attestation, and two independent weak sources can jointly exceed either alone. We therefore model a leaf’s evidence as a tuple and treat strength as a *partial* order.

```

1  evidence:
2    kind: fuzz_campaign          # the class label (a coarse prior, not the whole story)
3    scope: AdapterObservedDeltaAccounting  # which guarantee it actually establishes
4    coverage:
5      token_behaviors: [fee_on_transfer, rebasing, false_return, no_return, callback]
6      paths: [deposit, withdraw, redeem]
7    assumptions: {max_supply: bounded, gas_limit: 30M}  # the conditions under which it holds
8    artifact: test/AdapterInvariant.t.sol
9    issuer: internal             # vs third-party audit firm, protocol team, oracle operator
10   reproducibility: deterministic_from_commit_hash
11   expiry: commit_hash          # invalidated by any change to the attested code

```

Strength is then the product partial order on (scope $\supseteq$, coverage $\supseteq$, weaker-assumptions, higher-reproducibility, independent-issuer): $e_1 \succeq e_2$ only when e_1 dominates e_2 on every axis. Two leaves may be *incomparable* (a deep proof of a narrow property vs. a shallow check of a broad one), which is the honest situation; the class label is a coarse prior used only to break ties and to flag obviously-weak leaves (a critical need resting on a *doc-claim*). Along a critical path, confidence is bounded by the greatest lower bound of the evidence on that path *when it exists*; when the evidence items are incomparable, the ledger must *report the incomparability* rather than collapse them into a fabricated single score. We state this model as a deliberately *semi-formal* engineering device, not a lattice-theoretic theorem; over-formalising trust would be its own overclaim.

9.5 Economic needs, with a worked template

The economic-need class (Section 3.5) requires at least one concrete template to be more than a concept. The canonical oracle-seam economic property is that manipulation is unprofitable:

$$\forall \Delta p, T : \text{Cost}_{\text{manip}}(\text{oracle}, \Delta p, T) > \text{EV}_{\text{extract}}(\text{protocol}, \Delta p, T),$$

i.e. the cost of moving the price by Δp over horizon T exceeds the value extractable from the protocol under that move. This is not a pure trace property: it is quantitative and relative to an economic model that must be named as a leaf.

```

1  property:
2    id: E_ORACLE_001
3    name: ManipulationCostExceedsExtractableValue
4    modal_class: economic_safety
5    trace_layer: economic
6    seam_type: oracle_seam
7    expression: "Cost_manip(delta_p, horizon) > Extractable_value(delta_p, horizon)"
8    parameters:
9      liquidity_sources: [pool_A, pool_B, cex_arb]
10     oracle_window: twap_30min
11     max_flash_liquidity: estimated_bound
12     liquidation_bonus: 0.08
13     collateral_factor: 0.80
14     verification_modality: [economic_model, scenario_simulation]

```

```

15 | threat_tags: [T3, T2]
16 | evidence:
17 |   class: economic_assumption      # weakest-but-named; not a code invariant
18 |   artifact: models/oracle_manipulation_model.ipynb

```

The framework’s role here is precise and modest: it does not validate the economic model, it makes the model an explicit, parameterised, named dependency whose failure is localised to one leaf rather than diffused through the system. If the liquidity bound is wrong, the ledger says exactly which leaf is wrong.

9.6 A minimal internal catalogue

So that the paper is self-contained even if the companion ontology [36] is never opened, Table 10 gives twelve canonical properties with their classification. This is the catalogue the case study draws on; it is small by design — the claim is the *schema*, not exhaustiveness.

Property	Modal class	Layer	Seam type	Threat	Modality
ExactReceivedAmount	safety	evm-local	ABI	T5,T1	invariant fuzz
BalanceConservation	safety	evm-local	ABI	T5	invariant fuzz
NoUnexpectedCallback	safety	tx-context	callback	T1	static + call-graph
PreviewFaithfulness	safety	evm-local	accounting	T5	invariant fuzz
SharePriceMonotonicity	safety	evm-local	accounting	T2	invariant fuzz
InflationResistance	safety	evm-local	accounting	T5,T2	proof + fuzz
AuthorizedUpgradeOnly	safety	governance	authority	T4	static + formal
StorageLayoutStability	safety	governance	upgrade	T4	static (layout diff)
OracleFreshness	safety	economic	oracle	T3	static + config
OracleManipulationBound	economic-safety	economic	oracle	T3,T2	economic model
ReplayProtectedMessage	safety	cross-domain	cross-domain	T6	proof
BoundedLiquidationResponse	bounded-liveness	economic	operational	T7	economic + operator

Table 10. A minimal internal catalogue. Each row is a serialisable entry (Section 9.3) with its five-axis classification. The case study (Section 7) uses ExactReceivedAmount, PreviewFaithfulness, OracleFreshness, InflationResistance, AuthorizedUpgradeOnly, StorageLayoutStability, and BoundedLiquidationResponse; the rest round out the structuring seams of Table 9.

9.7 An algebra of ledger operations

A model and a workflow do not yet earn the word *calculus*; a calculus needs operations that combine and transform its objects. We define eight, each a total or partial operation on properties, guarantees, and ledgers, with a one-line semantics. They are exactly the moves the design rules (Section 6) and the case study perform, now named.

refine(P, Q) holds iff $\llbracket P \rrbracket \subseteq \llbracket Q \rrbracket$ (P is stronger than Q); the entailment underlying every edge.

conflict(P, Q) holds iff $\llbracket P \rrbracket \cap \llbracket Q \rrbracket = \emptyset$; specification conflict (Definition 3.3a).

discharge(G, A) adds an edge $G \rightarrow A$ when **refine**(G, A); the core ledger step.

weaken(A) is *not* a discharge operation and must never be used to make a ledger pass. It is a *design-change* operation: the integrator’s relied-upon need A may be replaced by a weaker $A' \supseteq A$ only if the component using A is itself modified so that A' is sufficient for that component’s own guarantee. Absent that modification, weakening the need silently invalidates the very guarantee it supports; the operation therefore carries a proof obligation on the consumer, not a relaxation of it.

wrap($C, \text{Adapter}$) replaces component C 's guarantee by the adapter's constrained guarantee plus residual-assumption leaves (the case-study adapter, Section 7).

project(P, layer) restricts P to a trace layer (Table 1); a layer mismatch (need in tx-context, evidence in evm-local) is a detectable error.

bound(L, k, clock, F) converts an unbounded-liveness need L into the safety need $\Box(\phi \Rightarrow \Diamond_{\leq k}^{\text{clock}} \psi \mid F)$ (Proposition 3.3), emitting the fairness leaf F .

compose($\mathcal{D}_1, \mathcal{D}_2$) merges two ledgers; defined iff the union is acyclic and every cross-ledger need is discharged — which is precisely the hypothesis of Theorem 3.1, so composition preserves the soundness conclusion.

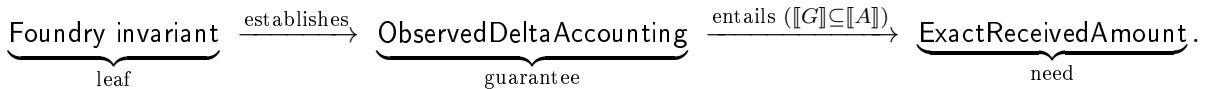
Proposition 9.1 (Composition preserves soundness). *If $\mathcal{D}_1, \mathcal{D}_2$ each satisfy the hypotheses of Theorem 3.1 and **compose**($\mathcal{D}_1, \mathcal{D}_2$) is defined (acyclic union, all cross-needs discharged), then the merged ledger satisfies them too, and the composed system satisfies the conjunction of both ledgers' safety guarantees.*

Proof. Acyclicity of the union is the hypothesis; every guarantee remains safety; every need (intra- or cross-ledger) has an entailing guarantee by assumption. The hypotheses of Theorem 3.1 hold for the union, giving the conjunction. $\square$

These operations are what make “seam calculus” more than a metaphor: the design rules are derived operations (**wrap** then **discharge**; **bound** then **discharge**), and Proposition 9.1 is the algebraic statement that the ledger composes. We do not claim this is a complete algebra (no equational theory, no normal forms); it is the minimal operation set that the method actually uses.

9.8 Reading the graph: proof-flow, not call-flow

Because dependency graphs are often drawn from consumer to provider, the orientation of Definition 3.5 deserves a one-edge picture. The chain for the case-study adapter is:



Edge direction is *proof-flow*, not call-flow: it runs from evidence, through the guarantee that evidence establishes, to the need that guarantee entails. The topological order therefore runs leaves $\rightarrow$ guarantees $\rightarrow$ needs, and acyclicity is well-foundedness of the proof, not of the call graph.

9.9 An operational-leaf template

Excluding operational adversaries (Section 3.3, the note after T8) from the formal model is a *scoping decision*, not a claim that they do not matter; for institutional infrastructure they are central. The framework admits them as a distinct leaf kind so the operational layer is not bolted on but typed:

```

1 operational_leaf:
2   id: OP_KEEPER_001
3   need: BoundedLiquidationResponse          # the bounded-liveness need it supports
4   guarantee_source: external_keeper_network
5   fairness_assumption: ">=1 honest keeper acts within k blocks"
6   evidence:
7     class: operator_assumption              # explicitly weak; named, not hidden
8     monitoring: alert_if_no_liquidation_within(k_blocks)

```



```

9   sla: keeper_uptime >= 99.9%
10  fallback: permissionless_liquidation_after(2k_blocks)
11  failure_mode: keeper_unavailability | unprofitable_after_gas (T7)
12  recoverability: degraded_not_catastrophic

```

Multisig unavailability, emergency-pause failure, oracle-operator downtime, frontend or RPC compromise, indexer corruption, solver misexecution, and governance bribery all instantiate this template, each becoming an **operator-assumption** leaf with a monitoring and fallback field. The ledger does not make these risks disappear; it gives each a typed, named home with an explicit fallback, which is the most an on-chain formal model can honestly do for an off-chain dependency.

Not every defect is equally dangerous; an audit needs prioritisation. Table 11 scores defects by combining the structural fault (missing edge, false/weak leaf, cycle) with the criticality of the need and the strength of the supporting evidence.

Severity	Ledger defect	Example
Critical	missing edge on a need with a direct asset-extraction path	fee-on-transfer vault share inflation
High	false or bypassable authority guarantee	UUPS upgrade-guard bypass
High	unbounded economic need	manipulable spot oracle, no cost bound
Medium	weak-evidence leaf (doc-claim/audit-only) under a critical need	solvency resting on a documentation claim
Medium	bounded-liveness need with an unverified fairness leaf	liquidation-keeper dependency
Low	specification or documentation ambiguity	unclear standard support
Informational	redundant edge	two guarantees discharge one need

Table 11. Severity scoring. The evidence ordering of Definition 3.5 feeds directly into severity: a need discharged only by a doc-claim leaf is a Medium defect even when the graph is well-formed, which is how the framework distinguishes a structurally complete from an epistemically strong ledger. Severity scoring is an audit *heuristic* — a function of asset exposure, exploitability, evidence weakness, centrality, and recoverability — and is deliberately *not* part of the soundness theorem; it orders attention, it does not certify safety.

9.10 What the calculus cannot detect

A defense-grade method must state its blind spots. By construction, the ledger is silent about: a need that is *absent* from the catalogue (an uncatalogued property cannot be undischarged); a leaf attestation that is *false* (a wrong Certora spec, a flawed audit, a lying oracle operator); an adversary *outside* $\mathcal{A}$ (e.g. governance bribery not modelled as T4, a novel cross-domain capability); a property *formalised over the wrong layer* (Table 1) — a proof about a thinner alphabet than the need requires; a gas/resource-exhaustion failure if gas is not carried in the trace alphabet; an unbounded-liveness failure if a fairness leaf is false; and a user-expectation violation that no catalogued property captures. Each blind spot corresponds to a specific place the framework *names* the gap — a missing catalogue entry, a weak leaf, an out-of-model capability, a wrong layer — which is the most it can honestly claim: not to remove these risks, but to make each one a labelled, locatable hole rather than a silent one.

9.11 Relation to existing audit methodologies

The ledger is not a replacement for existing practice but an organiser of its evidence. Table 12 positions it.

Method	What it catches / misses	Role in the ledger
Static analysis (Slither)	anti-patterns / economic & seam assumptions	generates candidate leaves
Fuzzing (Foundry, Echidna)	finite safety bugs / liveness, k -safety	validates adapter invariants
Formal verification (Certora, KEVM)	invariants, k -safety / wrong specs	discharges high-value leaves
Manual / competitive audit	design & context flaws / incomplete assumption graph	populates $\text{Need}(C, \rho)$
Economic audit	manipulation incentives / code-level auth	supplies economic-need leaves
Seam ledger (this work)	hidden cross-component assumptions / uncatalogued risks	organises all of the above as typed, evidenced leaves

Table 12. The ledger complements rather than competes with existing methods: each row’s output becomes a typed leaf with an evidence class, and the ledger’s contribution is the organising structure and the explicit residue, not a new verification engine.

The empirical security postures of the major stacks map onto the verification-modality axis v (Definition 8.1) and the gate (Theorem 8.2). Aave and Morpho publish explicit formal verification (Certora [14]), i.e. they use deductive, specification-based tools capable of expressing relational and system-level invariants — subject, as always, to the usual modelling, specification, solver, and trusted-computing-base assumptions, so “discharged” here means *relative to those assumptions*, never unconditionally; Morpho’s documented assurance combines audits, fuzzing, mutation testing, peer review and Certora proofs [35, 14]. Uniswap v4 emphasises a hook security *framework* — scoping the seam rather than the whole. Curve and Compound foreground audits and bug bounties, i.e. external attestations as ledger leaves. The most advanced teams combine the full ladder — unit tests, fuzzing, invariants, fork tests, static analysis, symbolic and deductive verification — which is exactly one tool per modal class.

Tool	Can help discharge (by modal class, Theorem 8.2)	Standing use
Foundry [15]	safety invariants; fork-tested integration seams	primary dev/test socle
Slither [16]	static safety patterns	systematic in CI
Echidna [17]	safety invariants via property fuzzing	targeted on critical primitives
Medusa	coverage-guided parallel fuzzing (safety)	heavy campaigns
Halmos [18]	bounded k -safety / relational via symbolic execution	targeted pathologies
Certora Prover [14]	system invariants, k -safety, authorisation, accounting	credit / upgrade / accounting cores

Table 13. Tooling mapped to obligation classes. The middle column states what each tool *can help discharge* under suitable modelling, not an unconditional guarantee: capability, model fidelity, the proof obligation, the assumptions supplied to the tool, and the trusted computing base are distinct concerns. Theorem 8.2 fixes only the necessary direction — single-trace tools (Foundry tests, Echidna, Medusa) cannot establish unbounded liveness or k -safety, for which relational/deductive tools (Halmos, Certora) are needed, themselves only under explicit modelling and fairness/bounded-horizon assumptions. No tool discharges unbounded liveness for an infinite-state system without such assumptions; no tool is omniscient, so a complete ledger needs several.

The recurrent seam failures and their discharges. The most frequent composition fault is not a missing `require` but a *contract mismatch* between abstractions: assuming a strictly standard ERC-20 that rebases, returns `false`, charges fees, or lacks `permit`; assuming `previewDeposit` is stable under an async or capped vault; assuming a hook is “just a callback”; assuming an EOA signature suffices where ERC-1271 is needed; assuming a proxy upgrade is transparent to the EIP-712 domain when a changed implementation alters validation; and now, post-Pectra, assuming `tx.origin`-based “human caller” checks survive EIP-7702 delegation. The discharges are repetitive and therefore automatable: feature-detect rather than assume; bound approvals in time and amount; normalise tokens behind adapters; pin blocks for fork tests; write conservation and solvency invariants; isolate callbacks behind explicit allow-lists; review the proxy/authz/oracle/adaptor seams separately; version modules explicitly; and treat module/adaptor registries as critical governance.

10 Related work

Compositional verification. The assume–guarantee discipline we use originates with Jones’s rely–guarantee [4] and Chandy and Misra’s parallel-program design [5], and the soundness subtlety we are careful about is Abadi and Lamport’s *Conjoining Specifications* / Composition Theorem [6], with de Roeper et al.’s monograph [7] as the standard reference for the circularity conditions. Our Theorem 3.1 is an application, not a new result; the contribution is its instantiation to EVM seams and the discharge-graph bookkeeping.

Hyperproperties. The safety/liveness decomposition is Alpern–Schneider [1, 2]; the k -safety classification and the limits on single-trace verification are Clarkson–Schneider [3]. We use these to justify the tool map (Table 13), not to extend the theory.

Smart-contract security and verification practice. The vulnerability taxonomy underpinning our threat model descends from Atzei, Bartoletti and Cimoli’s systematisation [8] and the community SWC registry [12]; the transaction-ordering capability T1 is the maximal-extractable-value surface of Daian et al. [11]. On the verification side, Certora’s Prover [14] (with published specifications for Aave, Compound and Morpho), the $\mathbb{K}$ framework’s KEVM semantics [13], and the static- and dynamic-analysis tools Slither [16], Echidna [17] and Halmos [18], with Foundry [15] as the integration substrate, constitute the deployed landscape; our role for each is fixed by the modality gate (Theorem 8.2) rather than by vendor claim. Property catalogues such as the SWC registry [12] are the informal ancestors of the catalogue in Definition 8.1; we differ in attaching an explicit modal class and an assume–guarantee ledger to each entry.

Provenance of the formal substrate. The five-axis catalogue, its typed relationship DAG, and its assume–guarantee ledger come from the author’s companion Blockchain Property Ontology [36]; this is a *companion* artifact of the same project, not independent external prior art in the way Alpern–Schneider, Clarkson–Schneider, or Abadi–Lamport are, and we flag the dependency as such. We instantiate it here rather than re-deriving it; the contribution here is the EVM instantiation, the two-fault correction (Definition 3.3), and the audit-reduction theorem with its honest decidability boundary (Theorem 8.3 and Corollary 8.4). The empirical spine — standardisation stages [27, 22, 26], library versions (OpenZeppelin v5.6.1 [19, 20]), and the EIP-7702 Pectra status [24, 25] — was cross-checked against the EIP repository, GitHub releases, and protocol documentation [33, 34, 35].

11 Limits and open questions

Four limits are inherent rather than editorial. *First*, by Theorem 8.2, hyperproperties (non-interference between positions, observational determinism of a router under reordering) lie beyond single-trace verification; the modelling cost of the relational tools they require is real and is not removed by the framework, only correctly assigned. *Second*, by Corollary 8.4, every leaf attestation $\text{Tr}(C) \subseteq \llbracket G \rrbracket$ is the undecidable model-checking problem in general; the framework localises this burden but cannot discharge it for free, and an attestation is only as strong as its issuer — an audit firm, a proof artifact, an oracle operator’s operational guarantee. *Third*, the catalogue’s semantics $\llbracket P \rrbracket$ must be *faithful* to the informal need; a mis-formalised property gives a sound proof of the wrong thing, and validating the formalisation against intent is a human obligation the calculus surfaces but does not automate. *Fourth*, and most fundamentally (Remark 8.2), the whole argument is relative to a chosen catalogue and threat model: a complete, acyclic, fully discharged ledger establishes the catalogued guarantees and nothing more, so a property no one wrote down or an attacker outside the model remains a live risk that the framework can name only once someone thinks of it. Open directions: (i) automatically extracting the need set $\text{Need}(C, \rho)$ from a standard’s prose and a component’s reference implementation; (ii) synthesising normalising adapters (R2) with a machine-checked conformance proof; (iii) deciding specification conflict (Definition 3.3(a)) for richer named fragments than safety LTL; (iv) earning the categorical reading — defining objects, morphisms, the guard, and a universal property so that seam discharge is literally a pushout, rather than the set-theoretic correspondence we state in Theorem 8.5; and (v) integrating the cross-domain/bridge seam (T6) into the calculus with finality and replay-protection obligations as first-class needs. The contribution we claim is correspondingly precise: for the safety fragment and relative to a stated catalogue and threat model, the security *argument* for a composed EVM system can be *stated* as a conjunction of discharged seam guarantees and *checked* as a decidable bookkeeping condition plus a finite, classified, tool-matched set of verification obligations — with the residual trust and the residual gaps named, not hidden. This makes the assumption ledger a mathematical object; it does not make security decidable, and it does not replace the socio-technical audit around it.

Scope, intended venue, and artifact roadmap. We position this work as a *formal engineering methodology* at the intersection of compositional verification and smart-contract security, and we are explicit that its natural first home is a venue valuing that intersection — the financial-cryptography and formal-methods-for-blockchain workshops (AFT, WTSC, FMBC) — rather than a pure formal-methods venue (which would demand a mechanised property language and tool artifact) or a pure security venue (which would demand an empirical incident-classification and detection evaluation). The novelty, restated once more for precision, is the instantiation layer of Section 9.7 and Section 9, not a new assume–guarantee theorem. The path to a full conference artifact is concrete and is the natural continuation of this paper: a machine-readable catalogue and ledger schema (Sections 9.3 and 9.6), a linear-time well-formedness checker over that schema, a worked example ledger (Table 6), and a Foundry invariant suite discharging the adapter leaf (Section 7). We present the present paper as the conceptual and structural foundation for that artifact. This working-paper version is deliberately comprehensive; a conference version would compress to roughly 12–18 pages by retaining the spine in the body — thesis and motivation, the *lface*/*Spec*/*Need* model, the two-fault distinction, the typed discharge graph, bounded liveness, the case study with the attack-pattern reconstruction, an operational-schema summary, and the limits — and relocating the reference material to an appendix: the full standards and platform dissection (Sections 4 and 5), the complete evidence and property schemas (Sections 9.3 and 9.4), the full minimal catalogue (Table 10), and the extended related work.

References

- [1] B. Alpern and F. B. Schneider. Defining liveness. *Information Processing Letters*, 21(4):181–185, 1985.
- [2] B. Alpern and F. B. Schneider. Recognizing safety and liveness. *Distributed Computing*, 2(3):117–126, 1987.
- [3] M. R. Clarkson and F. B. Schneider. Hyperproperties. *Journal of Computer Security*, 18(6):1157–1210, 2010. Preliminary version in *Proc. 21st IEEE Computer Security Foundations Symposium (CSF)*, 51–65, 2008.
- [4] C. B. Jones. Specification and design of (parallel) programs. In *Proc. IFIP Congress*, 321–332. North-Holland, 1983.
- [5] K. M. Chandy and J. Misra. *Parallel Program Design: A Foundation*. Addison-Wesley, Reading, MA, 1988.
- [6] M. Abadi and L. Lamport. Conjoining specifications. *ACM Transactions on Programming Languages and Systems*, 17(3):507–534, 1995.
- [7] W.-P. de Roever, F. de Boer, U. Hannemann, J. Hooman, Y. Lakhnech, M. Poel, and J. Zwiers. *Concurrency Verification: Introduction to Compositional and Noncompositional Methods*. Cambridge Tracts in Theoretical Computer Science 54. Cambridge University Press, 2001.
- [8] N. Atzei, M. Bartoletti, and T. Cimoli. A survey of attacks on Ethereum smart contracts (SoK). In *Principles of Security and Trust (POST 2017)*, LNCS 10204, 164–186. Springer, 2017.
- [9] S. M. Werner, D. Perez, L. Gudgeon, A. Klages-Mundt, D. Harz, and W. J. Knottenbelt. SoK: Decentralized Finance (DeFi). In *Proc. 4th ACM Conference on Advances in Financial Technologies (AFT ’22)*, 30–46, 2022. arXiv:2101.08778.
- [10] OWASP Foundation. OWASP Smart Contract Top 10. The 2026 (forward-looking) edition; its 2025 archive documents \$1.42B in losses across 149 incidents (SolidityScan Web3HackHub 2024), with Access Control the largest category by losses (\$953.2M). The 2026 list adds Proxy & Upgradeability as SC10. <https://owasp.org/www-project-smart-contract-top-10/>, accessed June 2026.
- [11] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels. Flash Boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *2020 IEEE Symposium on Security and Privacy (S&P)*, 910–927. IEEE, 2020.
- [12] SmartContractSecurity. SWC Registry: Smart Contract Weakness Classification and Test Cases. <https://swcregistry.io/>, accessed June 2026.
- [13] E. Hildenbrandt, M. Saxena, N. Rodrigues, X. Zhu, P. Daian, D. Guth, B. Moore, D. Park, Y. Zhang, A. Stefanescu, and G. Roşu. KEVM: A complete formal semantics of the Ethereum Virtual Machine. In *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*, 204–217. IEEE, 2018.
- [14] Certora. Certora Prover documentation: formal verification for smart contracts. <https://docs.certora.com/>, accessed June 2026.
- [15] Foundry contributors. Foundry: a portable, modular toolkit for Ethereum application development. <https://getfoundry.sh/>, accessed June 2026.

- [16] Trail of Bits. Slither: a static analysis framework for Solidity and Vyper. <https://github.com/crytic/slither>, accessed June 2026.
- [17] Trail of Bits. Echidna: a property-based fuzzer for Ethereum smart contracts. <https://github.com/crytic/echidna>, accessed June 2026.
- [18] a16z. Halmos: a symbolic testing tool for EVM smart contracts. <https://github.com/a16z/halmos>, accessed June 2026.
- [19] OpenZeppelin. OpenZeppelin Contracts (v5.6.1), MIT License. <https://github.com/OpenZeppelin/openzeppelin-contracts>, accessed June 2026.
- [20] OpenZeppelin. OpenZeppelin Contracts changelog (v5.x): ERC-4626 maxWithdraw/previewRedeem, ERC-7702 signer, ERC-7786 messaging, WebAuthn/P256 signers. <https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/CHANGELOG.md>, accessed June 2026.
- [21] OpenZeppelin. Doubling down on open-source tools and phasing out Defender (final shutdown 1 July 2026). <https://www.openzeppelin.com/news/doubling-down-on-open-source-and-phasing-out-defender>, 2026.
- [22] V. Buterin, S. Wilson, A. Dietrichs, and M. Garnett. EIP-7702: Set code for EOAs. Ethereum Improvement Proposals, <https://eips.ethereum.org/EIPS/eip-7702>, 2024 (final; deployed in Pectra, May 2025).
- [23] Security research on EIP-7702 delegation. EIP-7702 phishing and account-delegation attack surface. Preprint, <https://arxiv.org/abs/2512.12174>, 2025.
- [24] Ethereum Foundation. Pectra mainnet announcement. <https://blog.ethereum.org/2025/04/23/pectra-mainnet>, 2025.
- [25] ethereum.org. Pectra: EIP-7702 best-practice guidelines (delegate minimality, initialiser protection, tx.origin caveats). <https://ethereum.org/roadmap/pectra/7702/>, accessed June 2026.
- [26] A. Akhunov and M. Moelius. EIP-1153: Transient storage opcodes. Ethereum Improvement Proposals, <https://eips.ethereum.org/EIPS/eip-1153>, 2018.
- [27] JT Riley, Dillon, Sara, Vectorized, and Neodaoist. ERC-6909: Minimal Multi-Token Interface. Ethereum Improvement Proposals (Standards Track), <https://eips.ethereum.org/EIPS/eip-6909>, 2023.
- [28] OpenZeppelin. A novel defense against ERC-4626 inflation attacks; the donation/first-depositor attack and virtual-offset shares. OpenZeppelin, <https://www.openzeppelin.com/news/a-novel-defense-against-erc4626-inflation-attacks>, accessed June 2026.
- [29] OpenZeppelin. ERC-4626 documentation: the inflation attack and the virtual-offset mitigation. <https://docs.openzeppelin.com/contracts/5.x/erc4626>, accessed June 2026.
- [30] J. Offerijns, A. Sinelnikova, V. Arun, J. Santoro, F. Ali, and J. Martins. ERC-7540: Asynchronous ERC-4626 Tokenized Vaults. Ethereum Improvement Proposals, <https://eips.ethereum.org/EIPS/eip-7540>, 2023. (Mandates reversion of `preview*` on async flows.)
- [31] J. Offerijns et al. ERC-7575: Multi-Asset ERC-4626 Vaults. Ethereum Improvement Proposals, <https://eips.ethereum.org/EIPS/eip-7575>, accessed June 2026.
- [32] ERC-7535 authors. ERC-7535: Native Asset ERC-4626 Tokenized Vault. Ethereum Improvement Proposals, <https://eips.ethereum.org/EIPS/eip-7535>, accessed June 2026.

- [33] Uniswap Labs. Uniswap v4 documentation: PoolManager architecture, hooks, and flash accounting. <https://docs.uniswap.org/contracts/v4/overview>, accessed June 2026.
- [34] Aave. Aave V3 technical documentation: Pool, configurator, aToken, debt tokens, and risk parameters. <https://aave.com/docs/developers/smart-contracts>, accessed June 2026.
- [35] Morpho. Morpho Blue and Vaults V2 documentation: immutable markets, IOracle, adapters, caps, and curator permissions. <https://docs.morpho.org/>, accessed June 2026.
- [36] Z. Boudi (TKNFRA). Blockchain Property Ontology (BPO): a faceted catalogue of blockchain correctness properties with five classification axes, a typed relationship DAG, and an assume-guarantee ledger. Open-source ontology, <https://github.com/TKNFRA/blockchain-property-ontology>, accessed June 2026.